[CSED233] Data Structure

Priority Queue and Heap

Seung-Hwan Baek

POSTECH

Queue and Priority

- We learned queue
- FIFO (First-In First-Out) list
- Similar to the top in the stack, we have the front and the rear

$$Q = < a_1, a_2, \dots, a_n >$$

- Operations: Enqueue, Dequeue, …

How to incorporate priority for queue?

Priority Queue

- Priority queue H consists of a set of elements (organized by priority)
- Each element x has a priority $p(x)$ (also called importance or key)
  - Not necessarily unique
- Supports the following operations:
  - $\boldsymbol{Insert(x, H)}$ – arbitrary element insertion
  - $\boldsymbol{DeleteMin(H)} = Min(H) + Delete$
    - Delete elements in the order of priority

Priority Queue's Implementation

- Straightforward ways

	$Insert$	$DeleteMin$
Normal queue	$O(1)$	$O(n)$
Unsorted linked list	$O(1)$	$O(n)$
Sorted linked list	$O(n)$	$O(1)$

Quiz

- Why we have the following time complexities?

	$Insert$	$DeleteMin$
Normal queue	$O(1)$	$O(n)$
Unsorted linked list	$O(1)$	$O(n)$
Sorted linked list	$O(n)$	$O(1)$

Priority Queue's Implementation

- Can we do better than these?

	$Insert$	$DeleteMin$
Normal queue	$O(1)$	$O(n)$
Unsorted linked list	$O(1)$	$O(n)$
Sorted linked list	$O(n)$	$O(1)$

Heap

- Tree-based data structure that satisfies the heap property
- Time complexity becomes $O(\log n)$ for both $Insert$ & $DeleteMin$

	$Insert$	$DeleteMin$
Normal queue	$O(1)$	$O(n)$
Unsorted linked list	$O(1)$	$O(n)$
Sorted linked list	$O(n)$	$O(1)$

Quiz

- Why $O(\log n)$ for both $Insert$ & $DeleteMin$ is better than below?

	$Insert$	$DeleteMin$
Normal queue	$O(1)$	$O(n)$
Unsorted linked list	$O(1)$	$O(n)$
Sorted linked list	$O(n)$	$O(1)$

Types of Heaps

- Binary heap
- Binominal heap
- Fibonacci heap
- 2-3 heap
- etc.

- This lecture focuses on binary heap

Binary Heap

- Tree is a (binary) min heap if it satisfies the two properties:

(1) Complete binary tree (Structural property)
- Can be implemented in an array

(2) Min tree (Heap order property)
- $p(node) \leq p(children)$

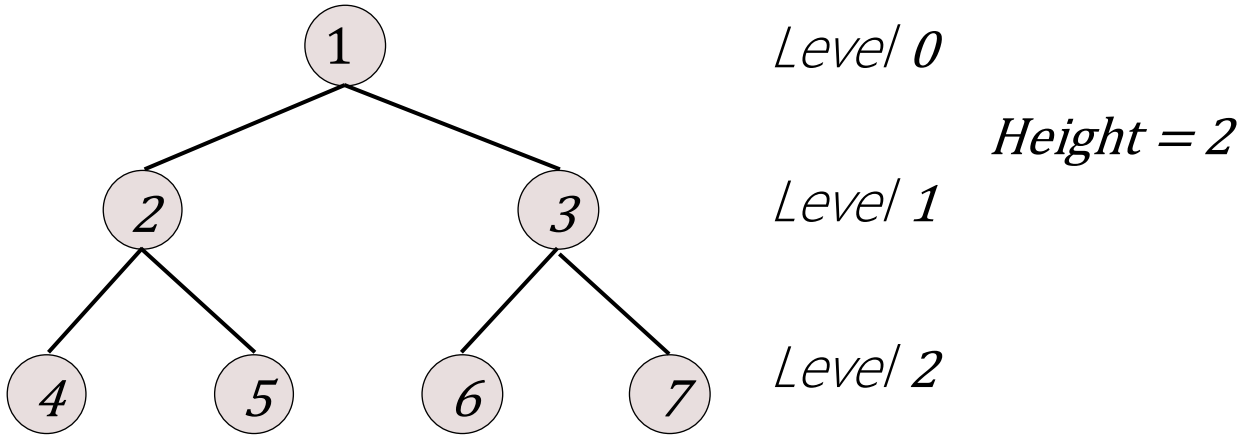
- Tree is a (binary) max heap if it satisfies the two properties:

(1) Same as above

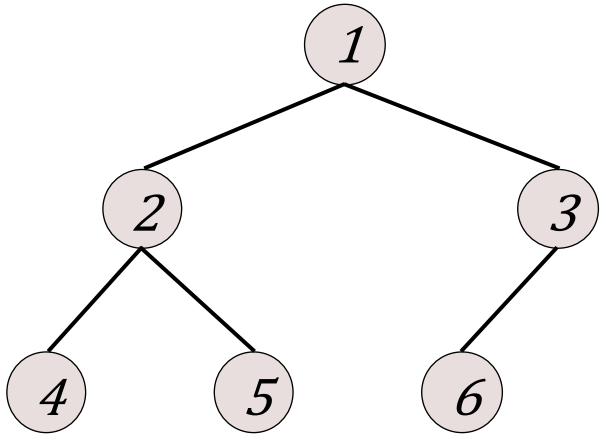
(2′) $p(node) \geq p(children)$

Recap: Complete Binary Tree

- Relaxed definition of a full binary tree
- A binary tree of height h is complete, if
  - All levels (possibly except h) are completely full
  - Level h (leaf level) is filled from left to right

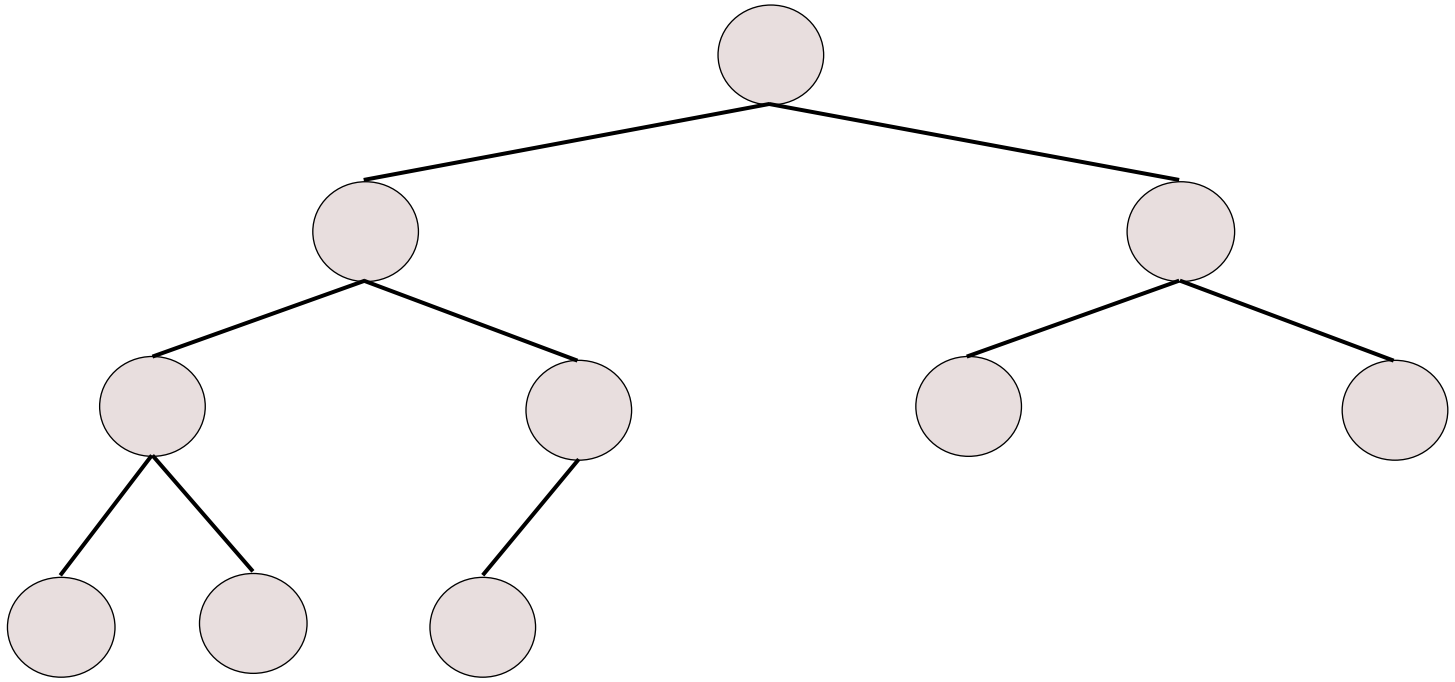


full binary tree with 7 nodes



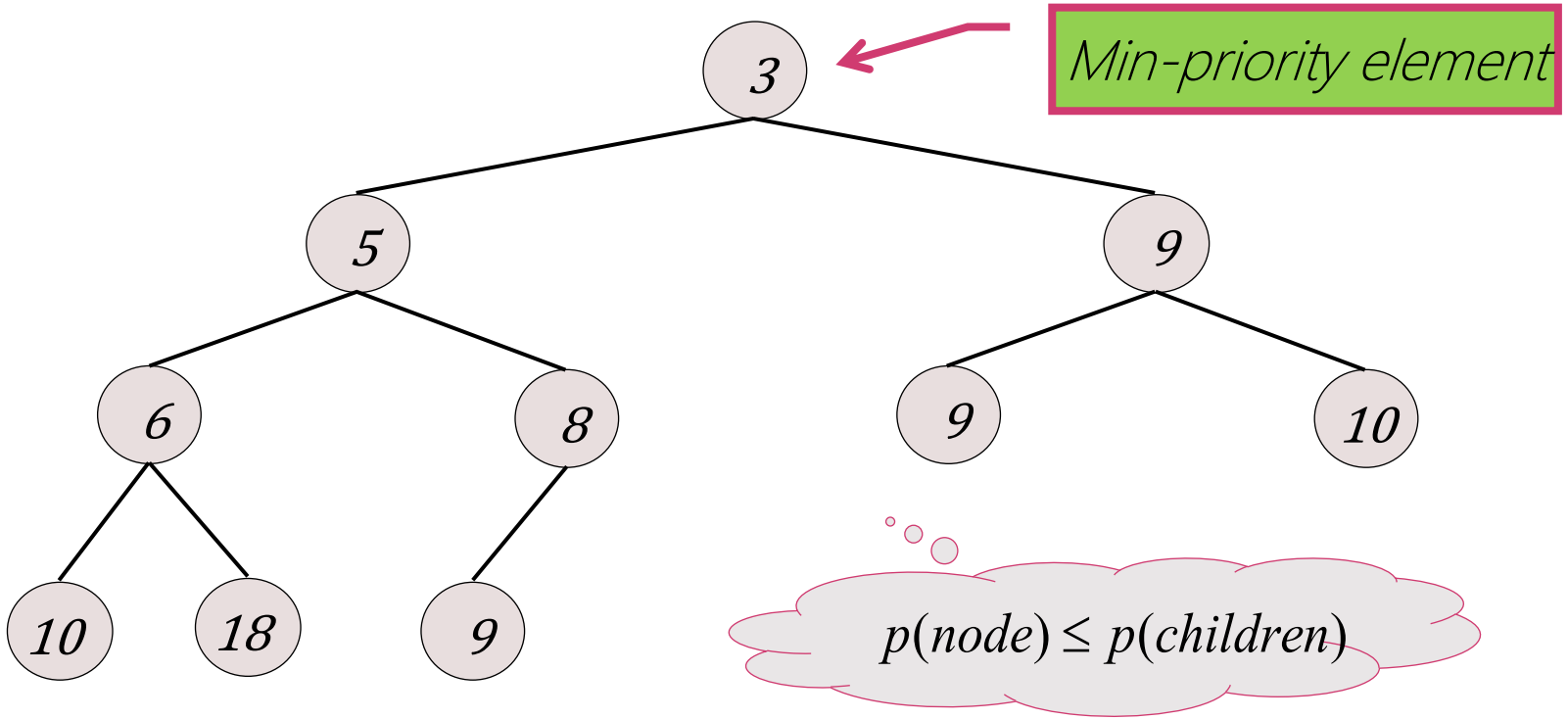
Complete binary tree with 6 nodes

Min-Heap: Structural Property



- Complete binary tree (CBT) with 10 nodes

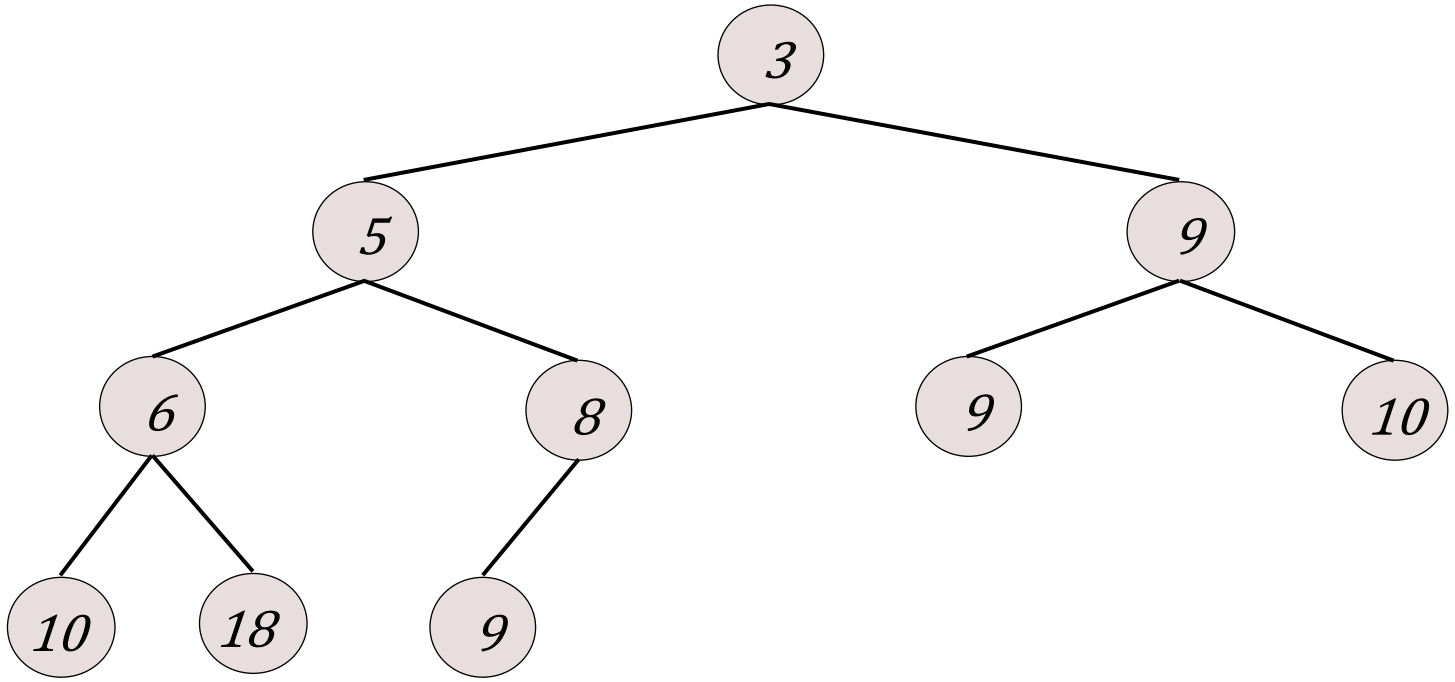
Min-Heap: Heap Order Property



- Complete binary tree (CBT), satisfying *min-heap order property*

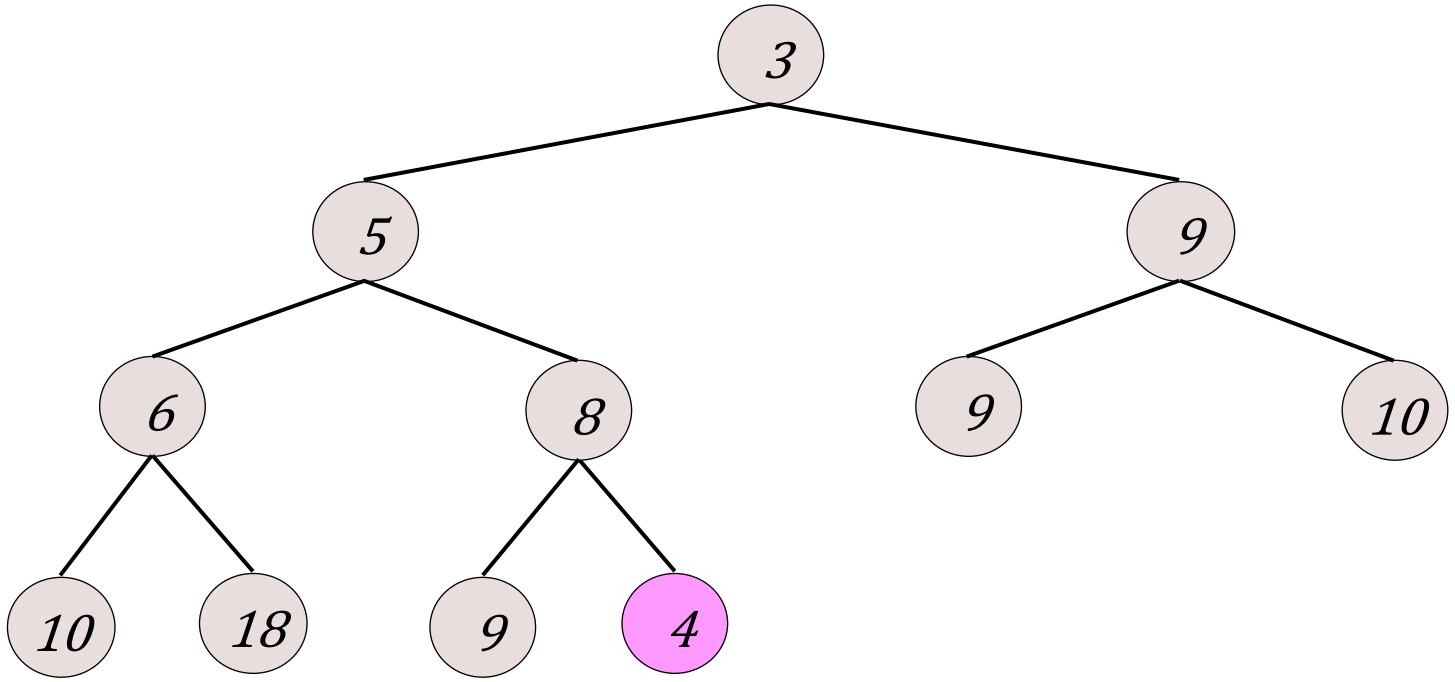
➔ Min-Heap

Insert into Min-Heap



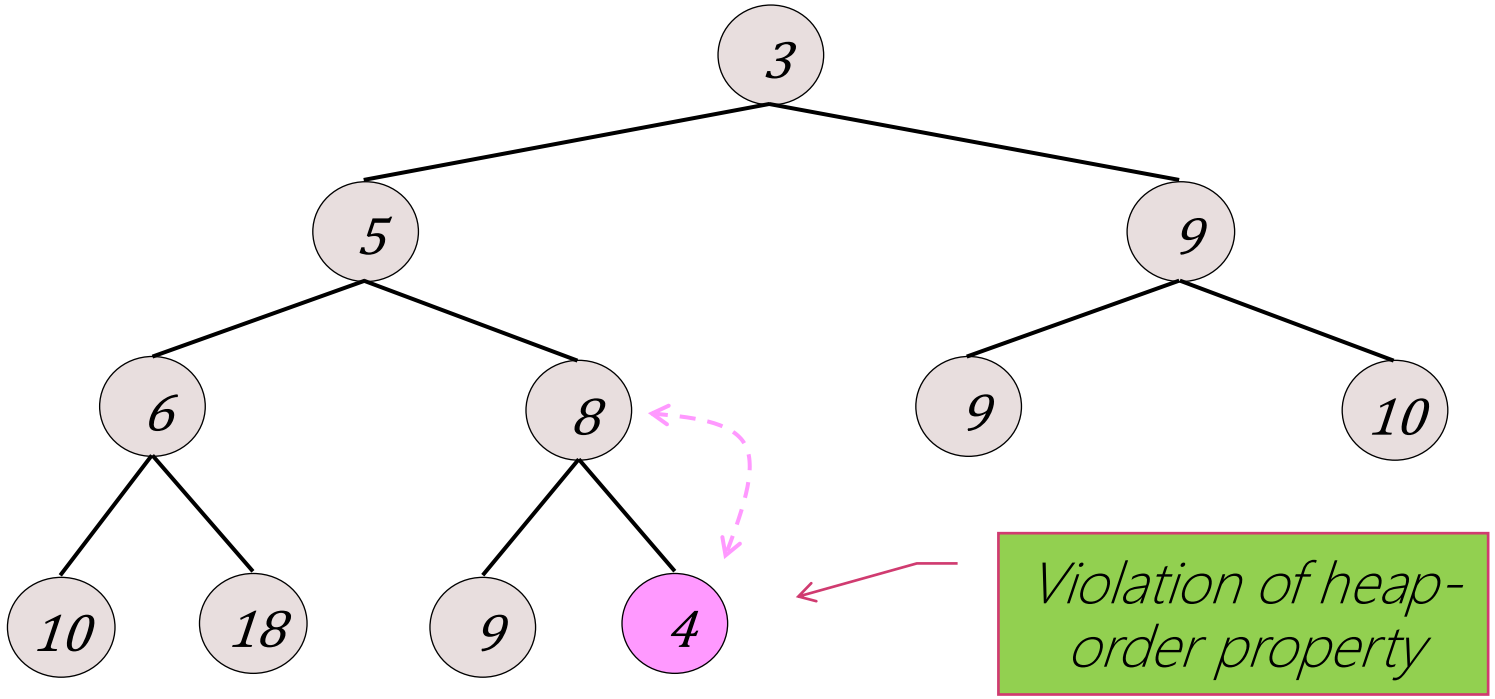
- Now, we want to insert a new element x into the heap H
  - $Insert(x, H), p(x) = 4$

Insert into Min-Heap: *Hole Creation*



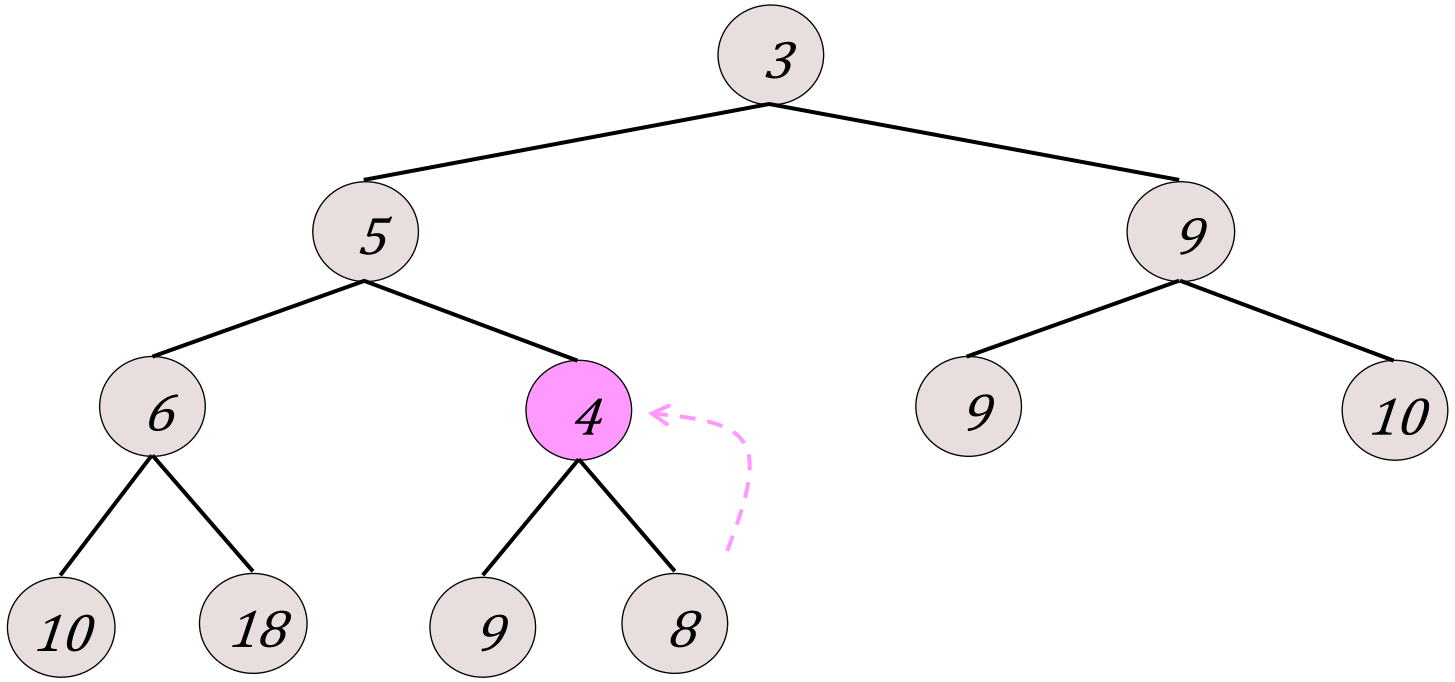
- Step-1: Create a hole & then store x in it
  - To satisfy the structural property (CBT), a new node must be added to the *rightmost* position of the *lowest* level

Insert into Min-Heap: *Up-Heap Bubbling*



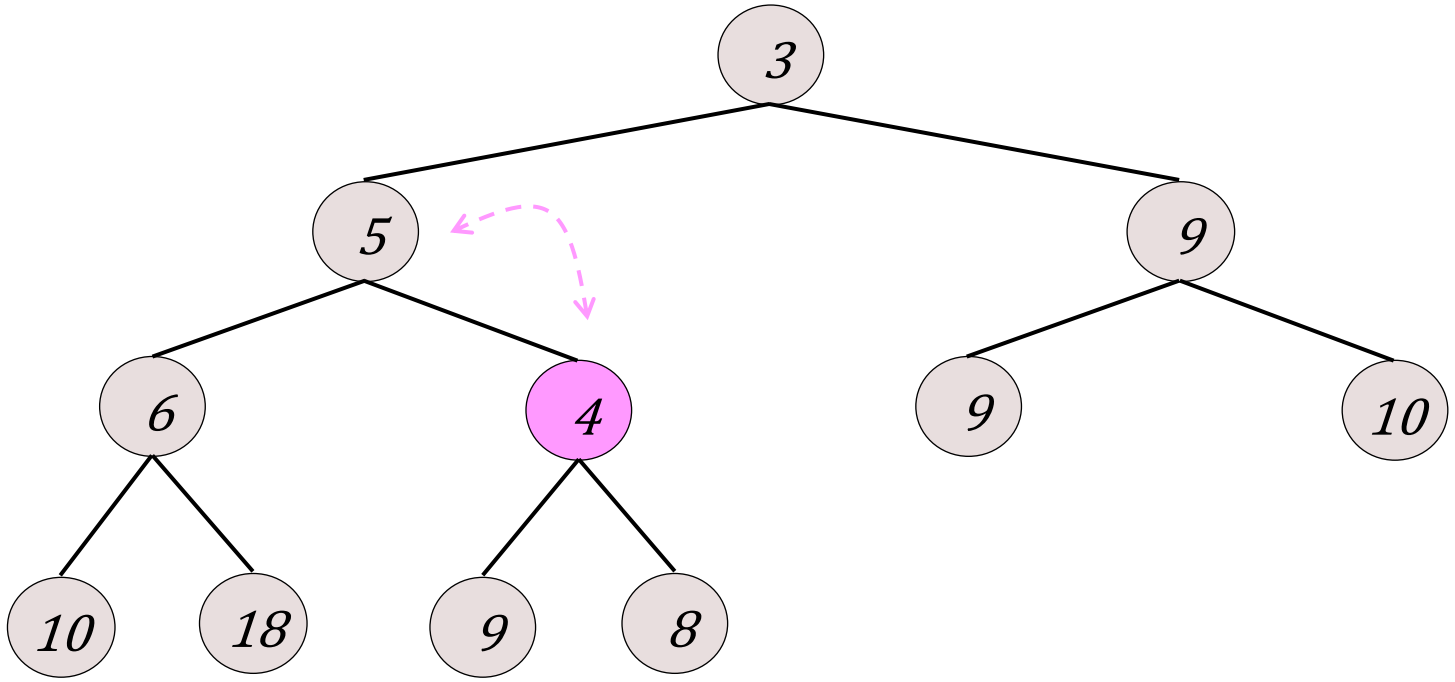
- Step-2: Restore the heap order property
  - Compare $p(x)$ with $p(parent)$ & swap them if necessary
    - Upward movement by swapping 4 & 8

Insert into Min-Heap: *Up-Heap Bubbling*



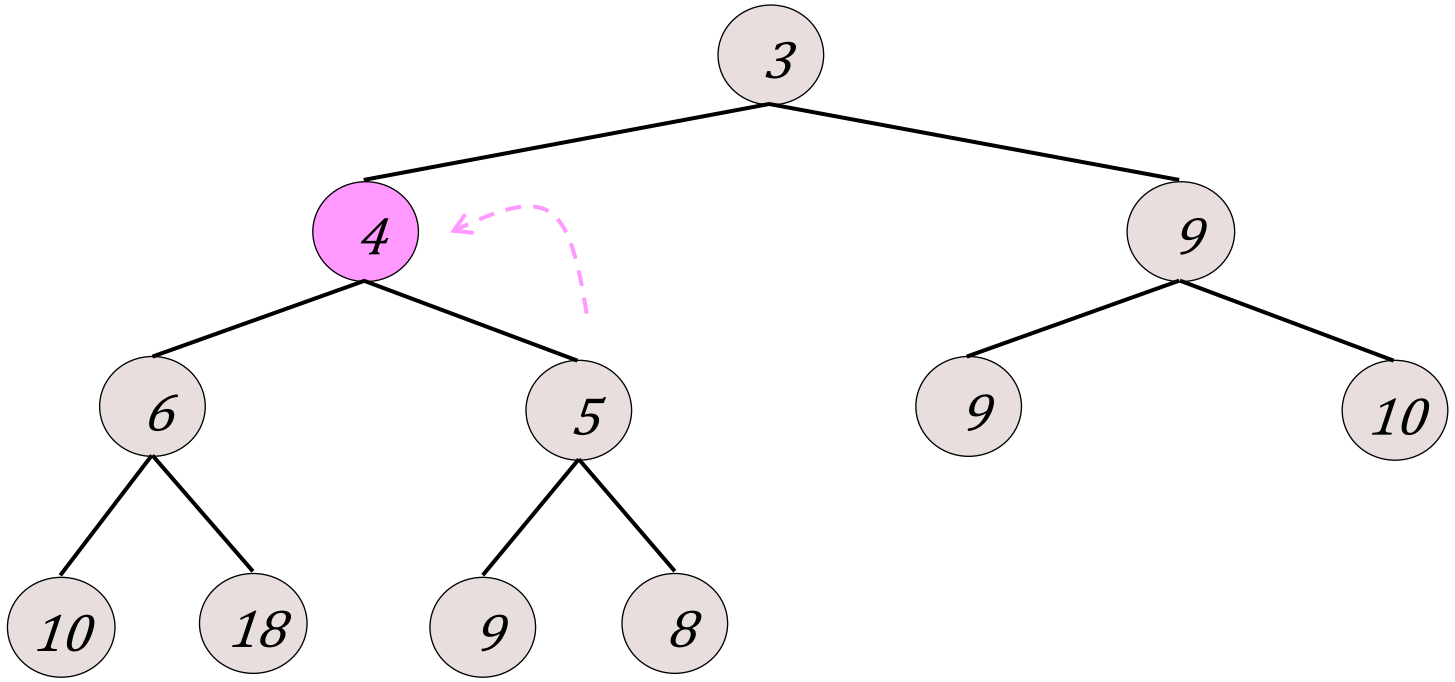
- Step-2: Restore the heap order property
  - Compare $p(x)$ with $p(parent)$ & swap them if necessary
    - Upward movement by swapping 4 & 8
    - Called "*Up-heap Bubbling*"

Insert into Min-Heap: *Up-Heap Bubbling*



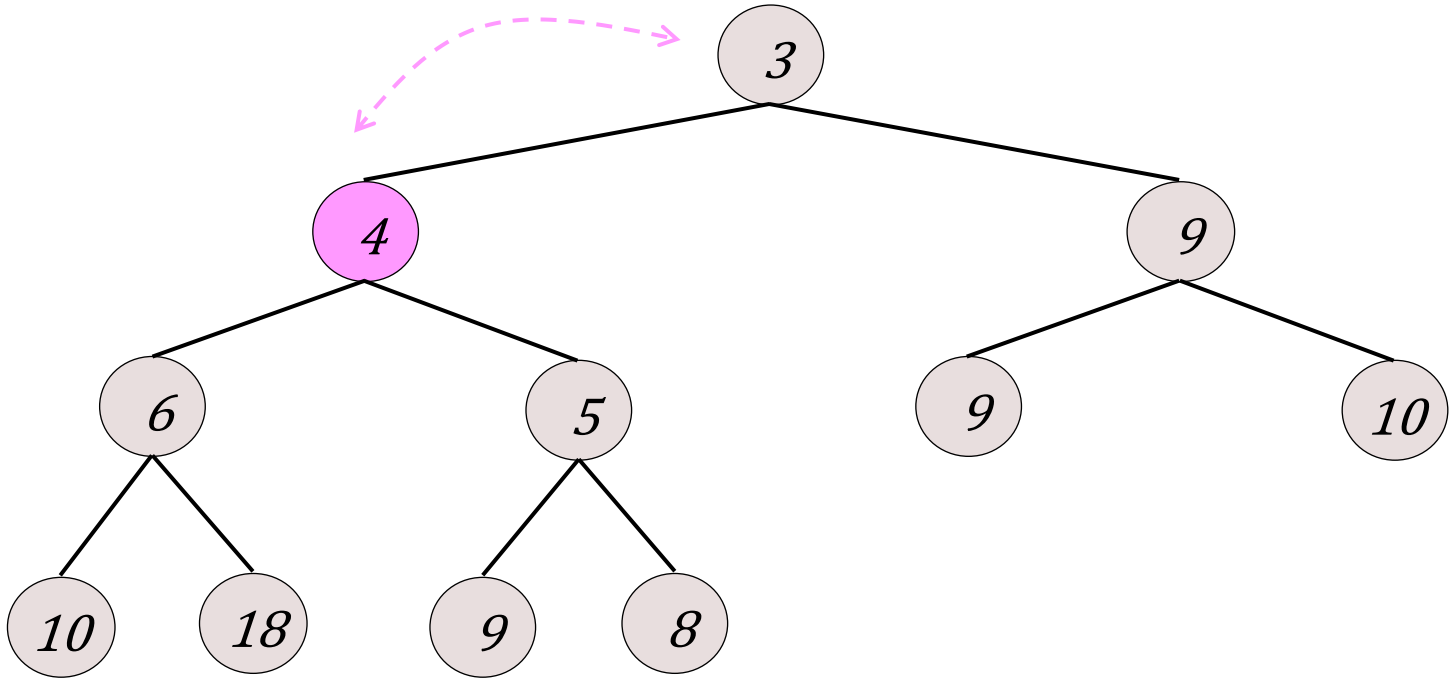
- Step-2: Restore the heap order property
  - Compare $p(x)$ with $p(parent)$ & swap them if necessary
    - Upward movement by swapping 4 & 5

Insert into Min-Heap: *Up-Heap Bubbling*



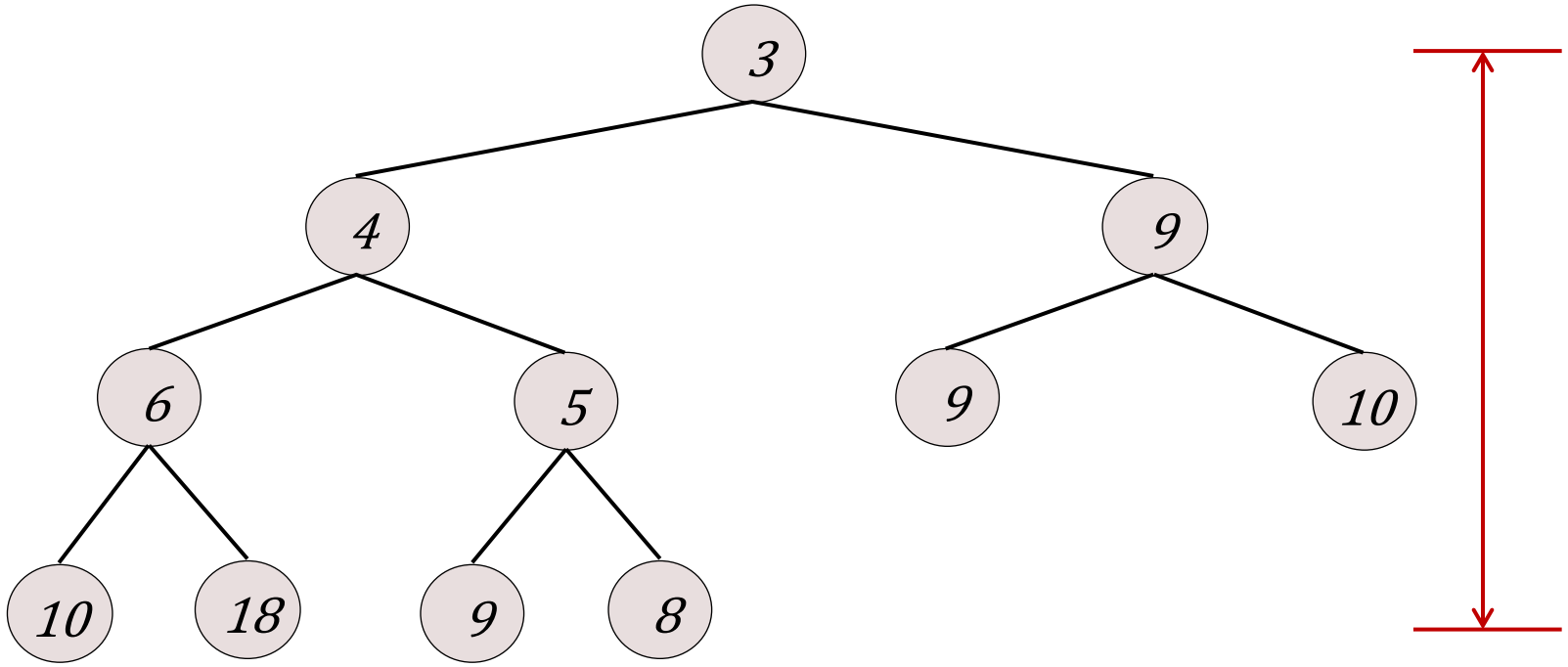
- Step-2: Restore the heap order property
  - Compare $p(x)$ with $p(parent)$ & swap them if necessary
    - Upward movement by swapping 4 & 5

Insert into Min-Heap: *Up-Heap Bubbling*



- Step-2: Restore the heap order property
  - Compare $p(x)$ with $p(parent)$ & swap them if necessary
    - No more up-heap bubbling
- Insertion completed

Insert into Min-Heap: Time Complexity



$$height(H) = O(\log(n))$$

- Time complexity of insertion?

$$= O(\log(n))$$

Quiz

- Which statement correctly describes the comparison rule used during up-heap bubbling in a min-heap?

A. Compare the new node with the smaller of its siblings.

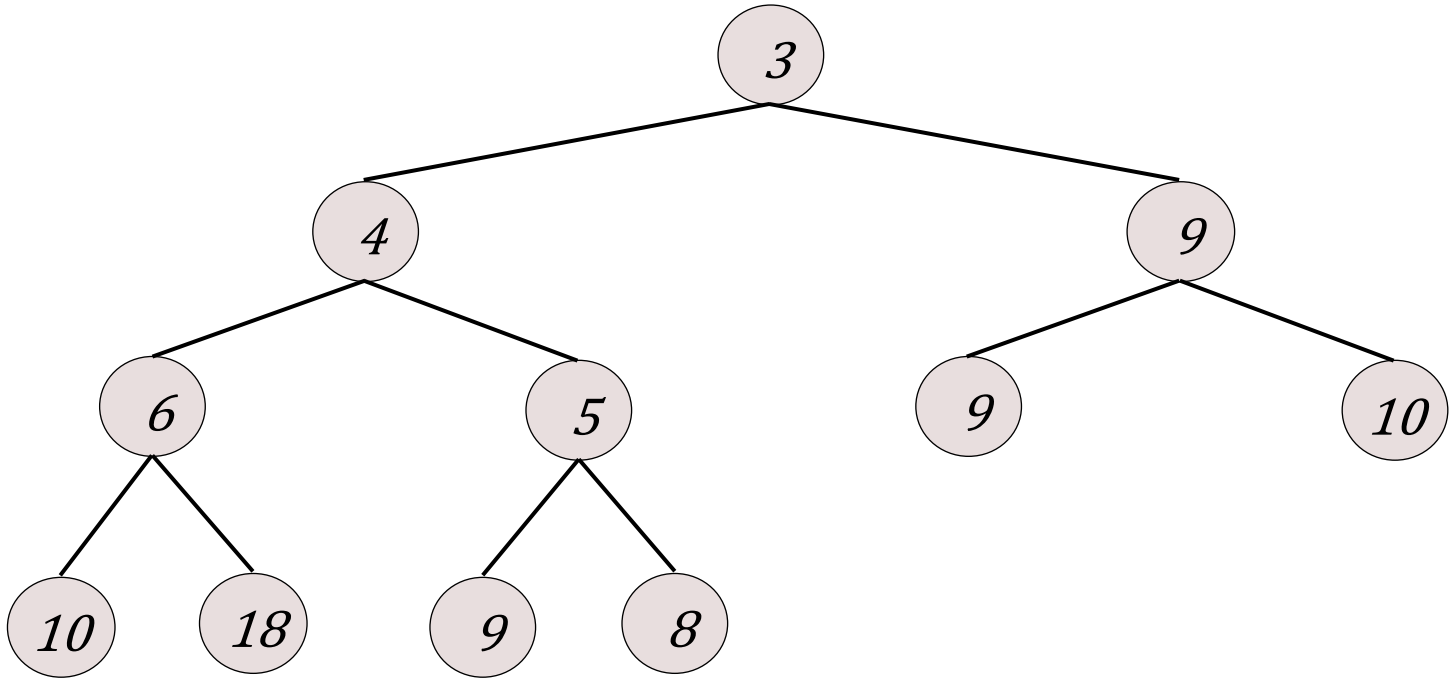
B. Compare the new node only with its parent; siblings are irrelevant.

C. Compare the new node with both siblings and parent.

D. Compare the new node with the root at every step.

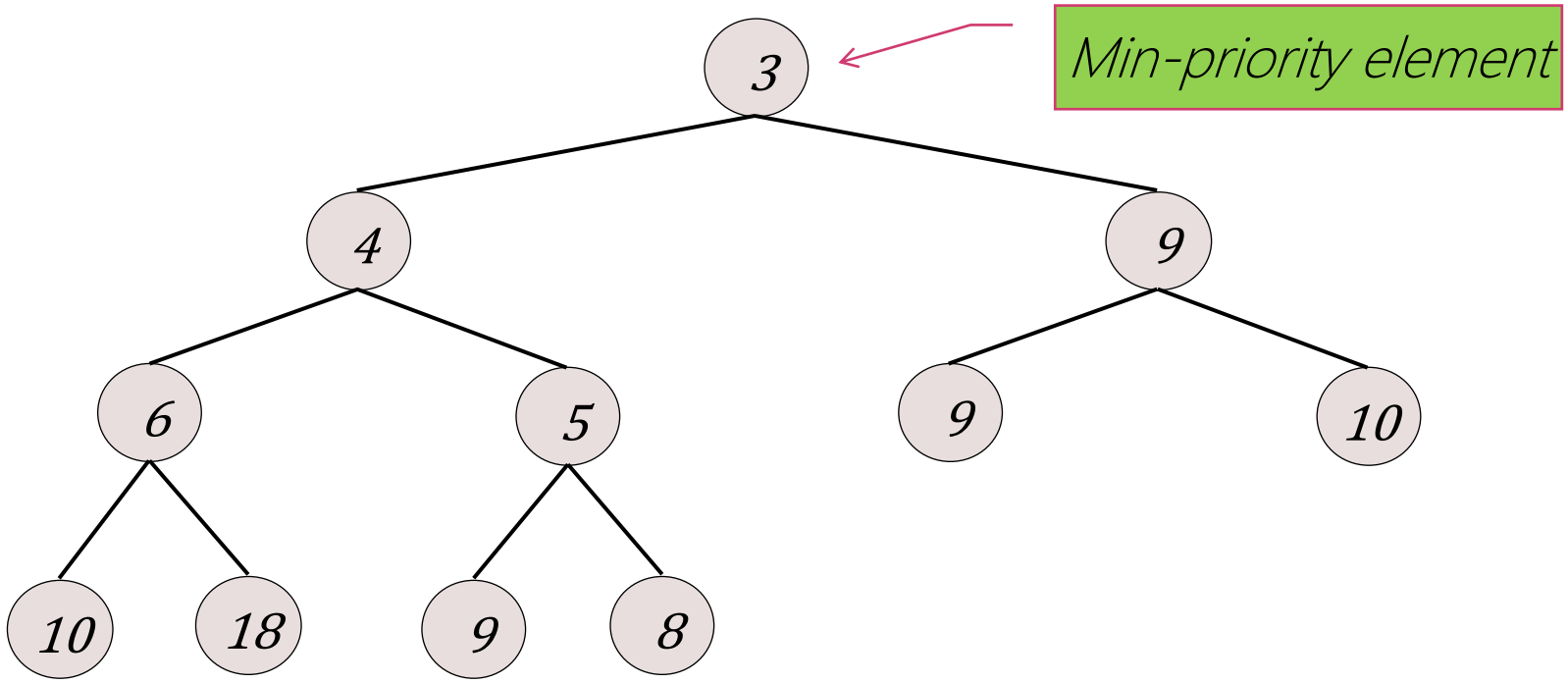
- Can you prove it?

$DeleteMin$ from Min-Heap



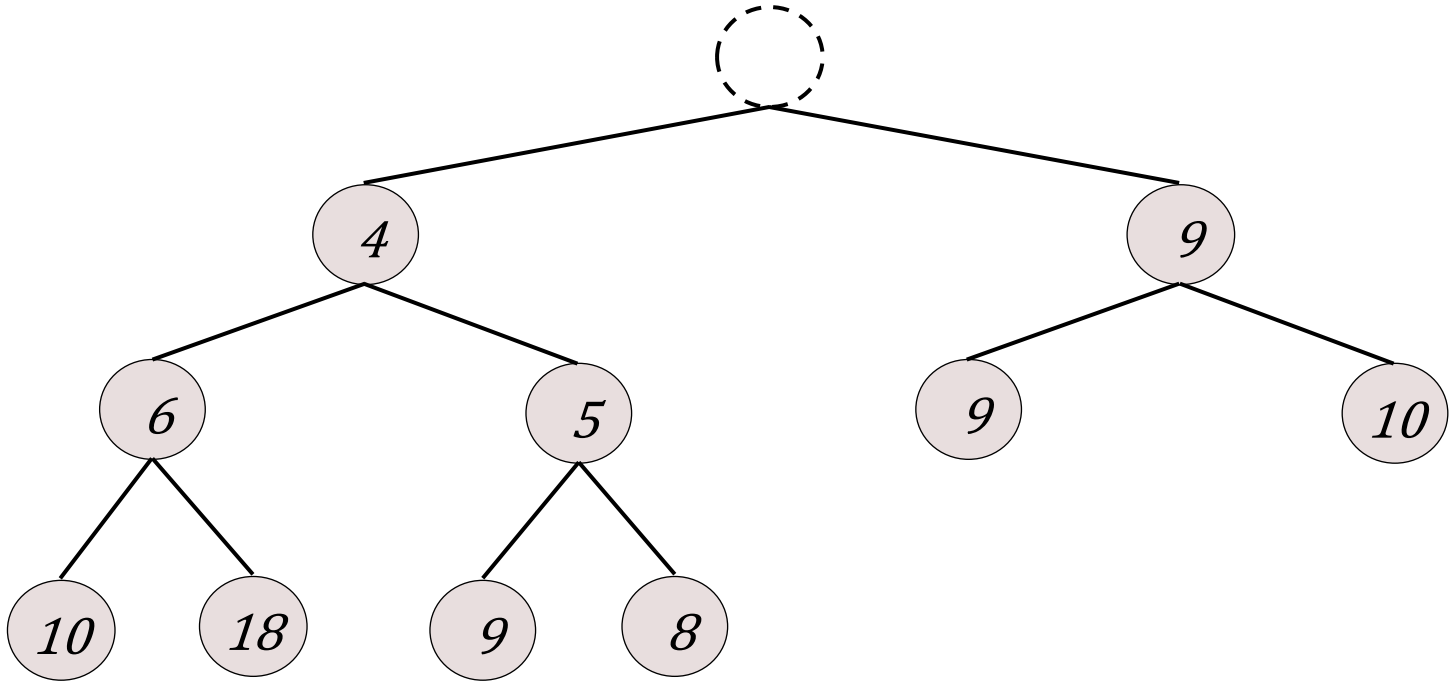
- We want to delete an element with the lowest priority
  - $DeleteMin(H)$

DeleteMin from Min-Heap



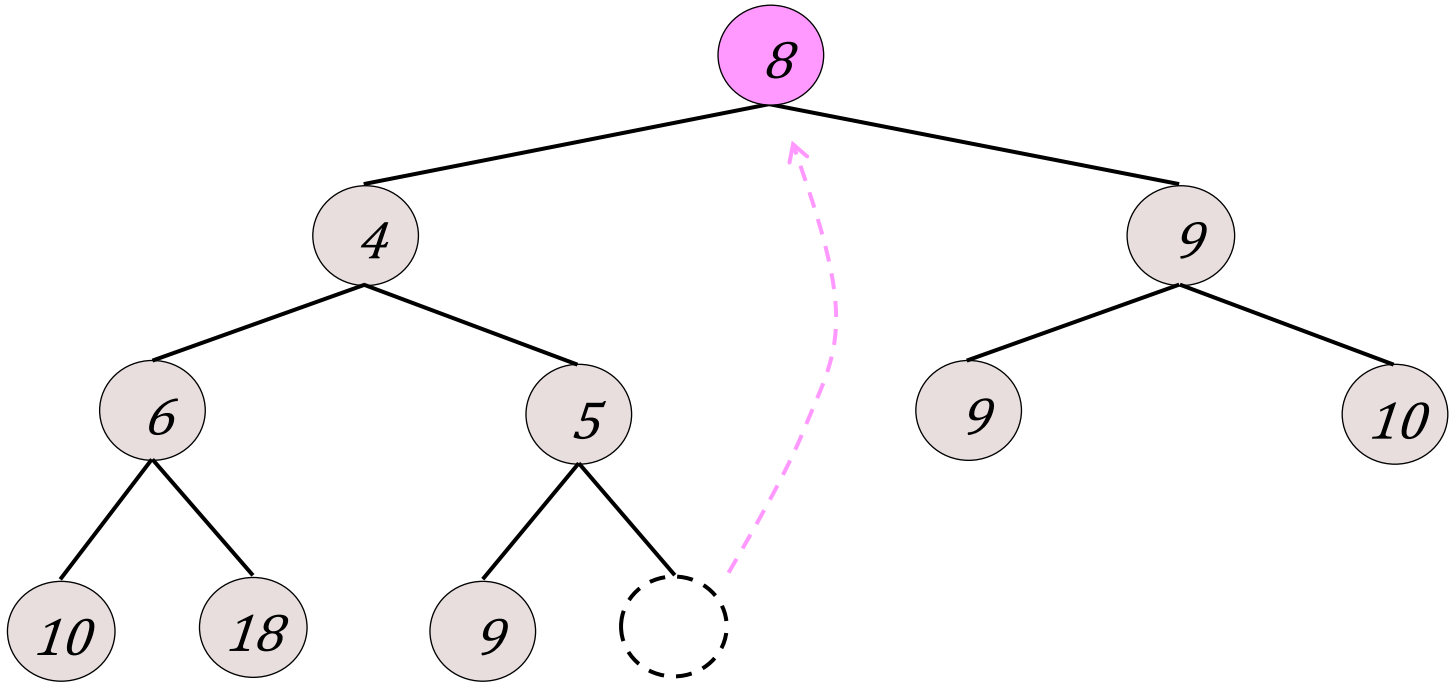
- Step-1: Remove the root & then move the last element to the hole

$DeleteMin$: Remove Root



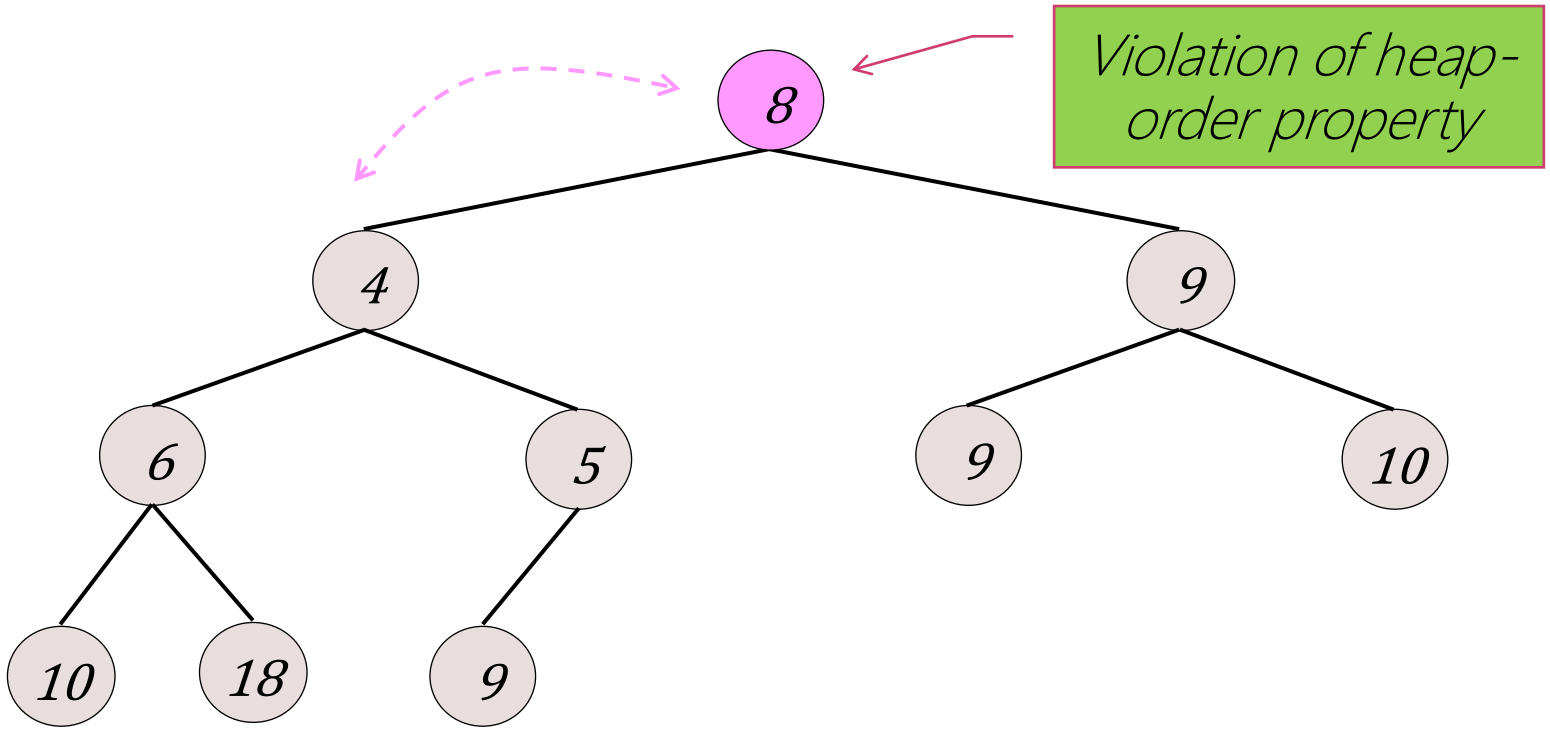
- Step-1: Remove the root & then move the last element to the hole

DeleteMin : Move Last One



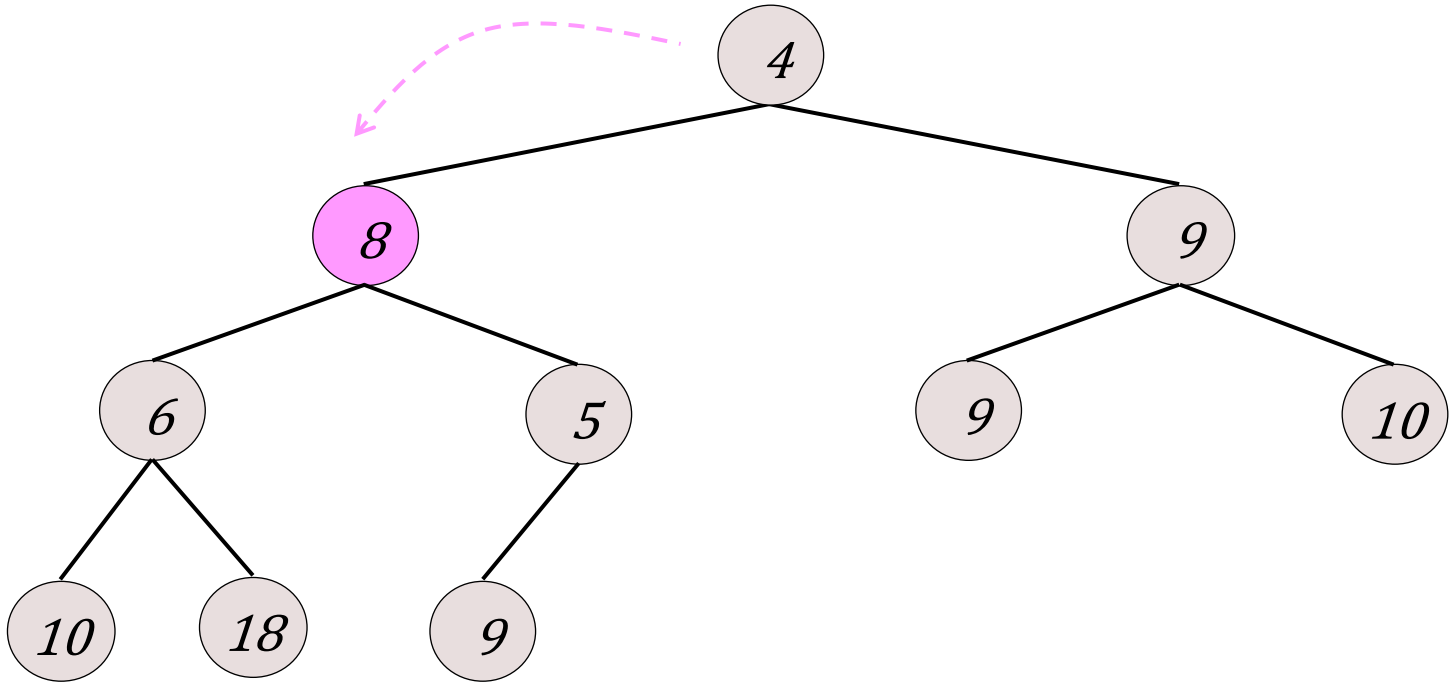
- Step-1: Remove the root & then move the last element to the hole

$DeleteMin$: Down-Heap Bubbling



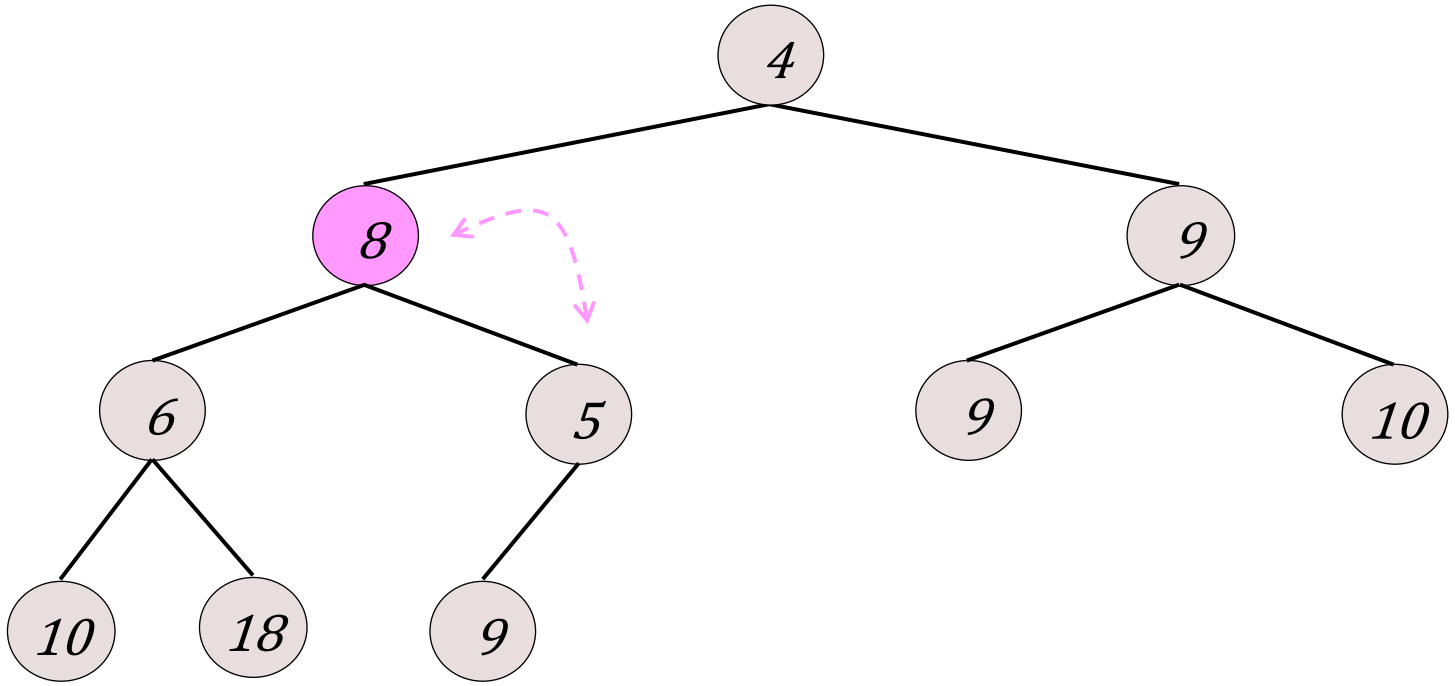
- Step-2: Restore the heap-order property
  - Compare $p(x)$ with $p(children)$ & swap them if necessary
    - Downward movement by swapping 8 & 4

$DeleteMin$: Down-Heap Bubbling



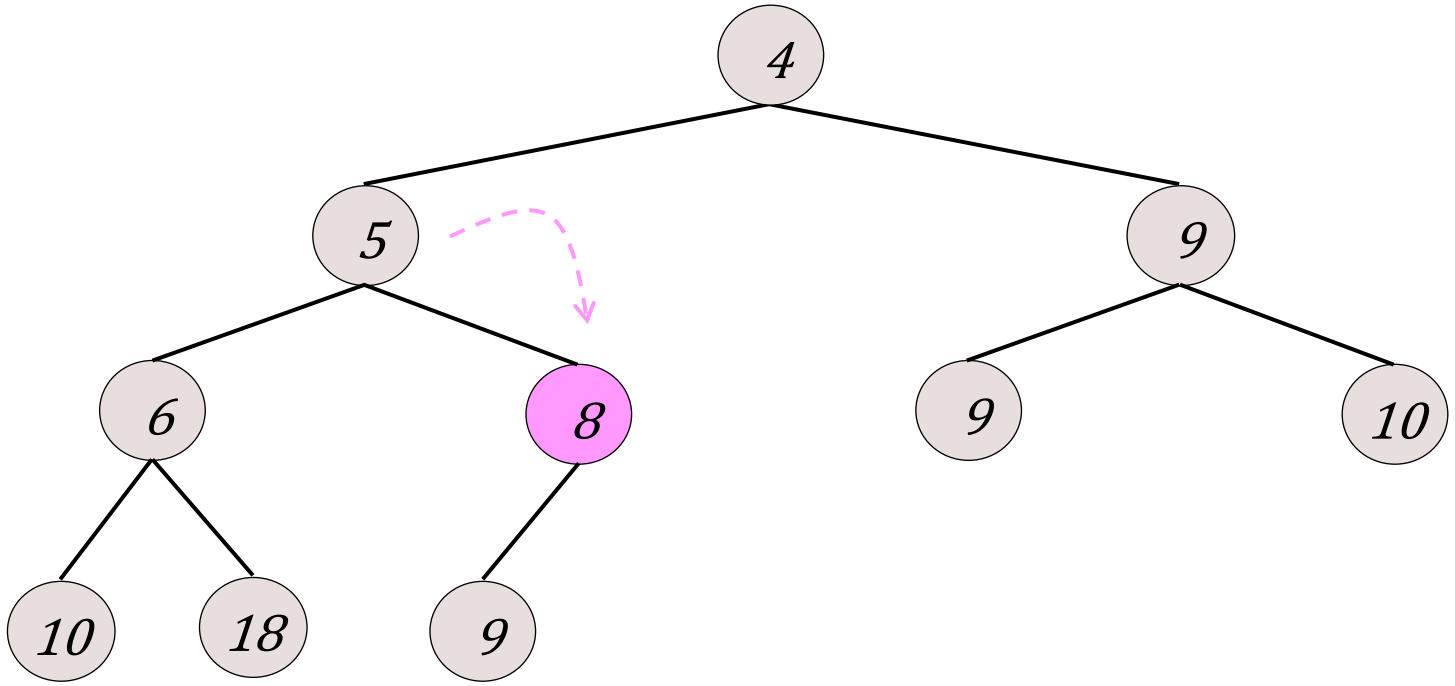
- Step-2: Restore the heap-order property
  - Compare $p(x)$ with $p(children)$ & swap them if necessary
    - Downward movement by swapping 8 & 4
    - Called "*Down-heap Bubbling*"

$DeleteMin$: Down-Heap Bubbling



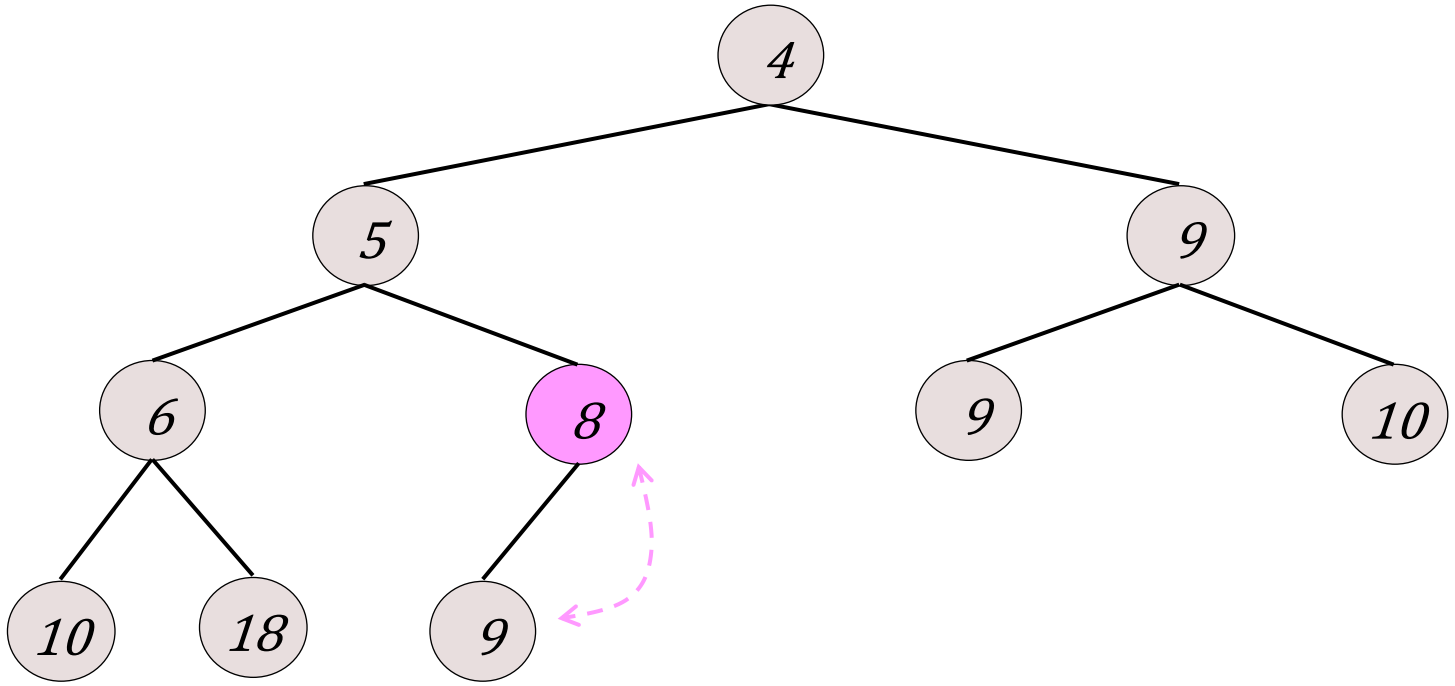
- Step-2: Restore the heap-order property
  - Compare $p(x)$ with $p(children)$ & swap them if necessary
    - Downward movement by swapping 8 & 5

$DeleteMin$: Down-Heap Bubbling



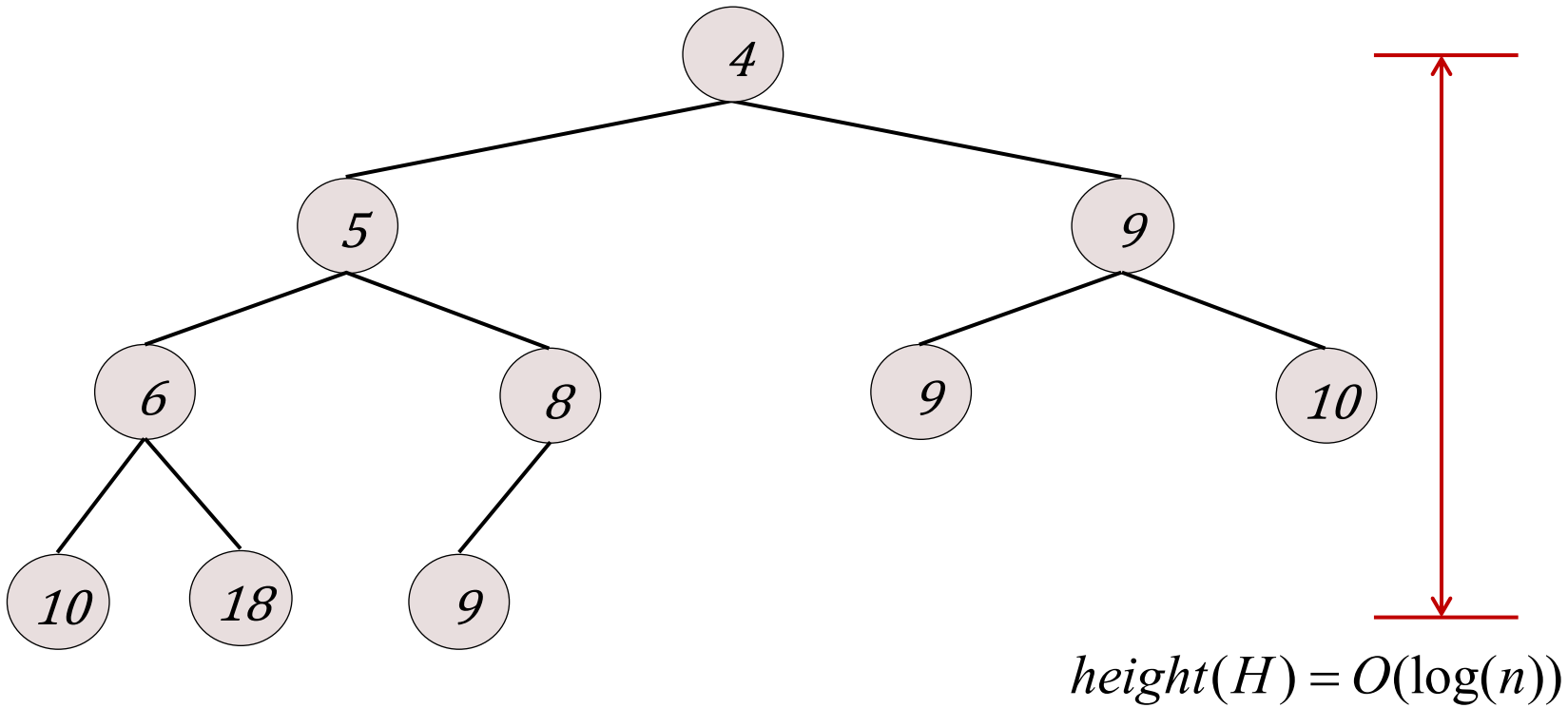
- Step-2: Restore the heap-order property
  - Compare $p(x)$ with $p(children)$ & swap them if necessary
    - Downward movement by swapping 8 & 5

$DeleteMin$: Down-Heap Bubbling



- Step-2: Restore the heap-order property
  - Compare $p(x)$ with $p(children)$ & swap them if necessary
    - No more down-heap bubbling
- $DeleteMin$ completed

$DeleteMin$: Time Complexity



- Time complexity of $DeleteMin$?

$$= O(\log(n))$$

Quiz

- Which statement correctly describes the comparison rule used during down-heap bubbling in a min-heap?

A. Compare the current node with both children and swap with the smaller child.

B. Compare the current node with its parent only.

C. Compare the current node with the larger child to maintain order.

D. Compare the current node with all nodes in the subtree.

- Can you prove it?

Quiz

- A binary min-heap must satisfy the heap order property
- Which of the following statements correctly describes this property?

A. Every parent node has a value less than or equal to its children.

B. Every level of the tree must be completely full.

C. The root must contain the largest value in the tree.

D. Every insertion must happen at the root.

Heap-Building Process

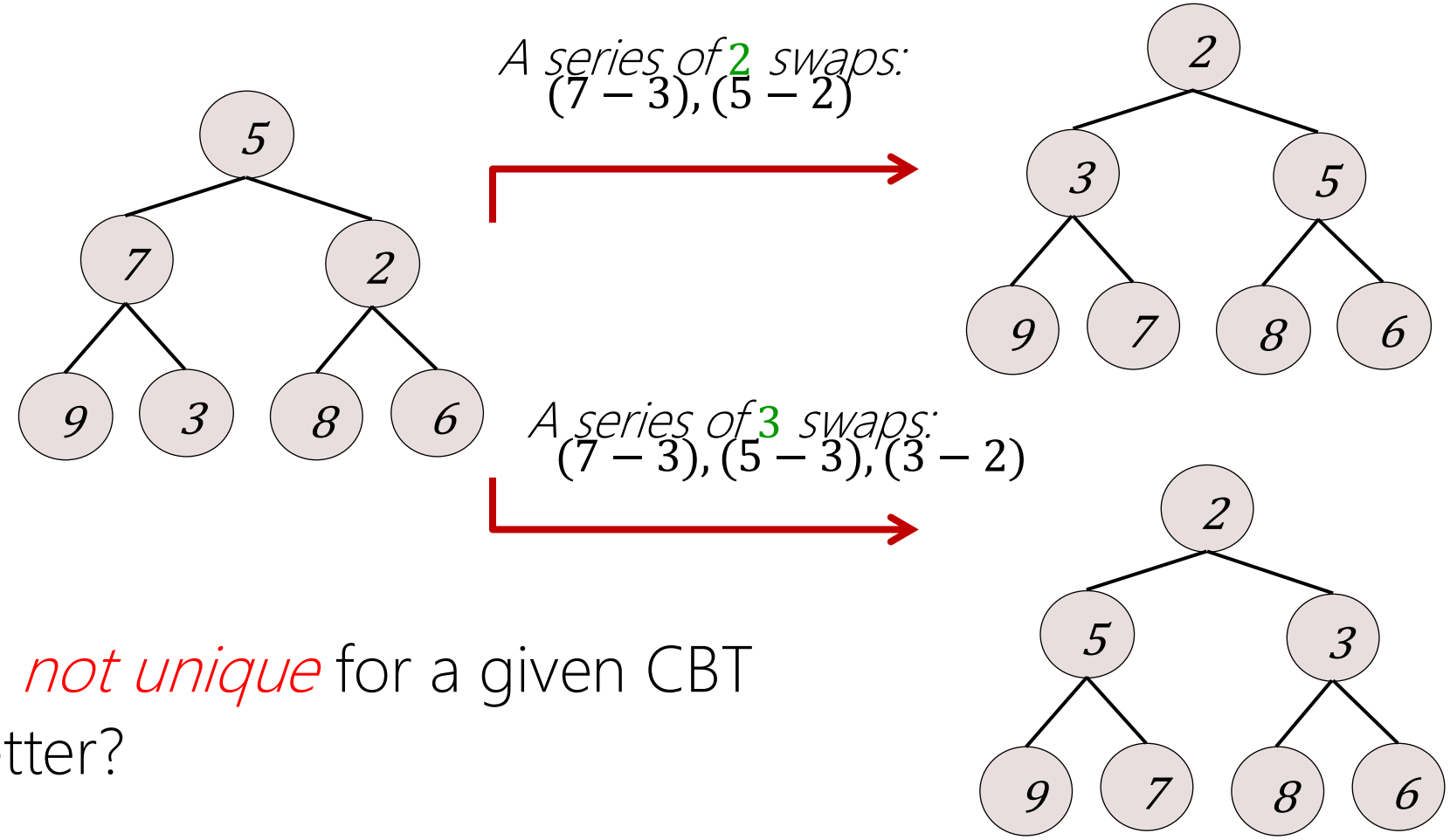
- Obvious way
  - Insert n elements one at a time
    - (i.e.) n successive insertions
  - Time complexity ?

    = $O(n \log n)$ in the worst case

- More efficient way
  - When all n elements are available in advance
  - Algorithm of $O(n)$ time:
  - How?

- Place n elements into an *array-based CBT* in any order
- *Heapify* the complete binary tree

Many Ways of Rearranging CBT



- The heap is *not unique* for a given CBT
- Which is better?

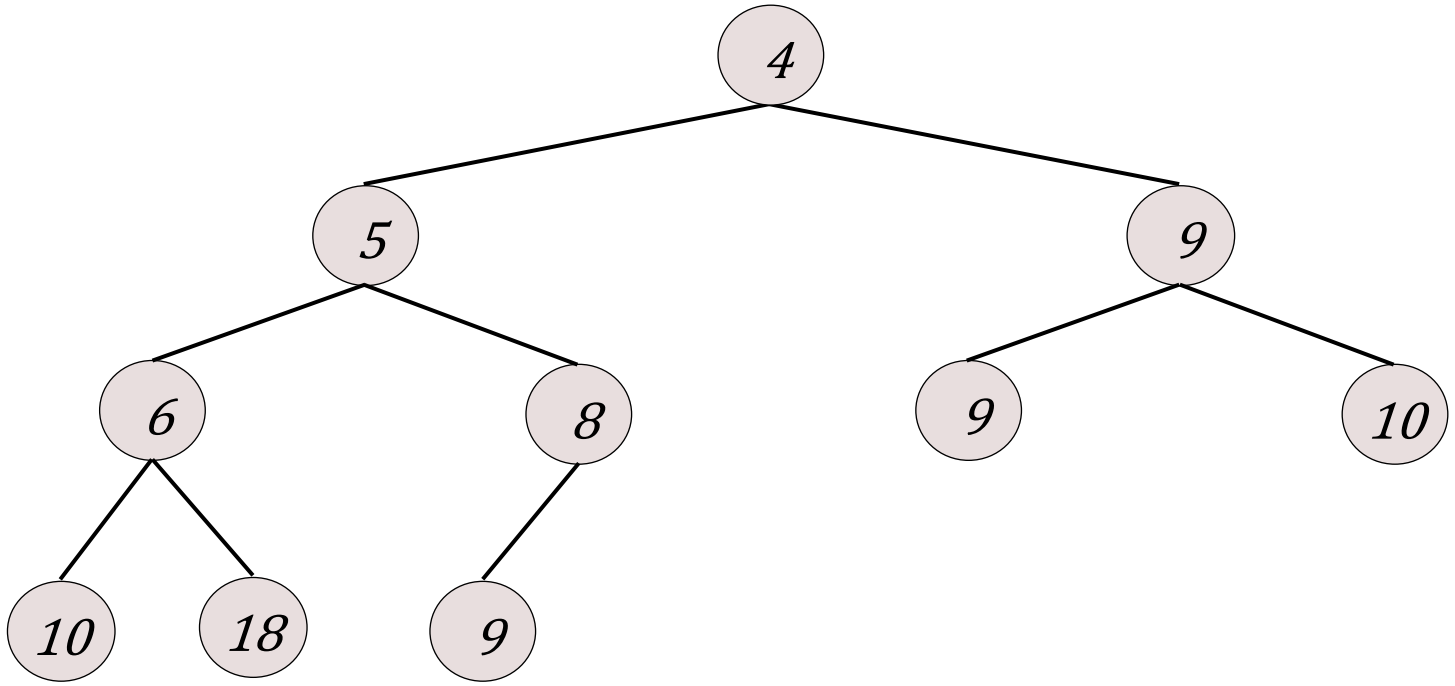
How to Heapify CBT

- How do we pick the *best* rearrangement

- Algorithm (from induction)
  - Input: Array-based CBT

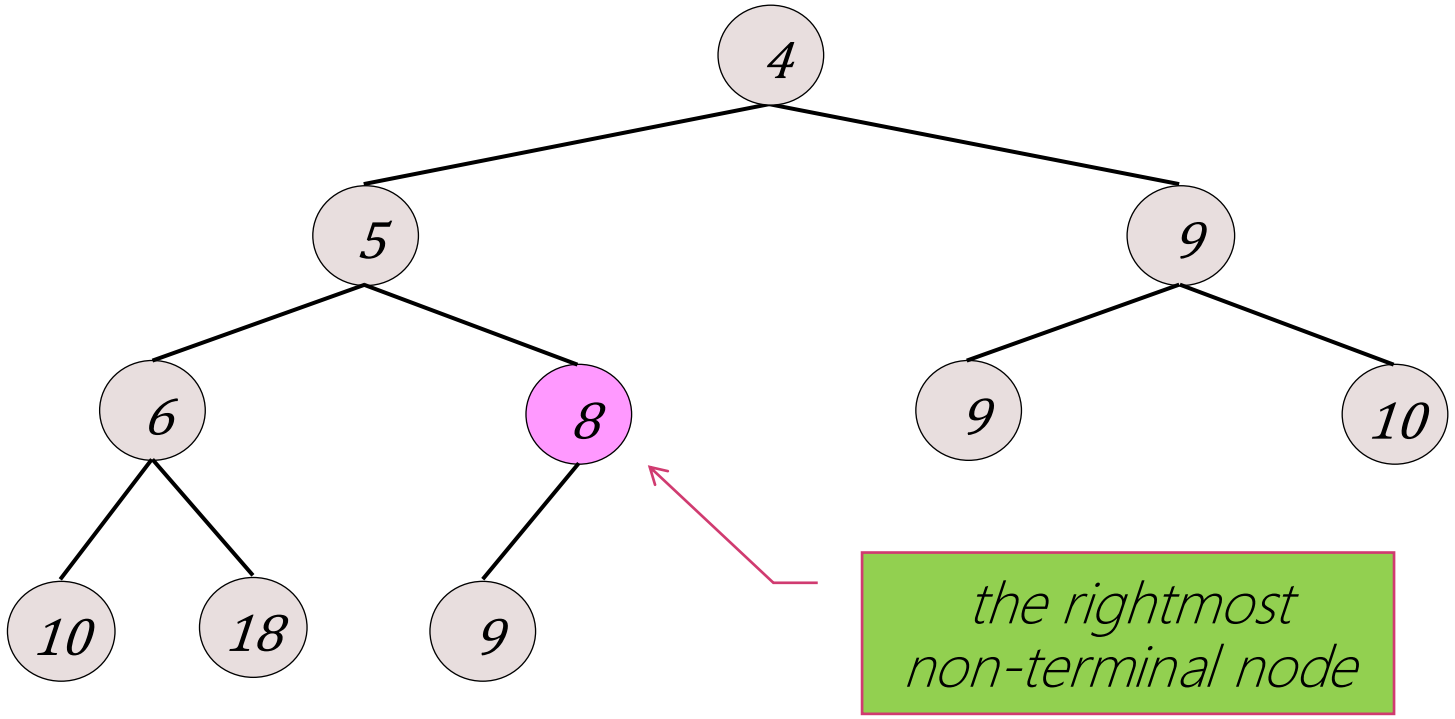
1) Start with the rightmost array position k that has a child
   (NB: the last non-terminal node at index $k = floor(\frac{n}{2})$), n is # of nodes
2) If the subtree rooted at k is not a min-heap
   - Rearrange the subtree (by pushing down k until it reaches a level where it is less than its children, or is a leaf node)
3) Repeat the step-2 at $k-1, k-2, \ldots, 1$ (from the high index of the array to the low index)

Example: Min-Heap → Max-Heap

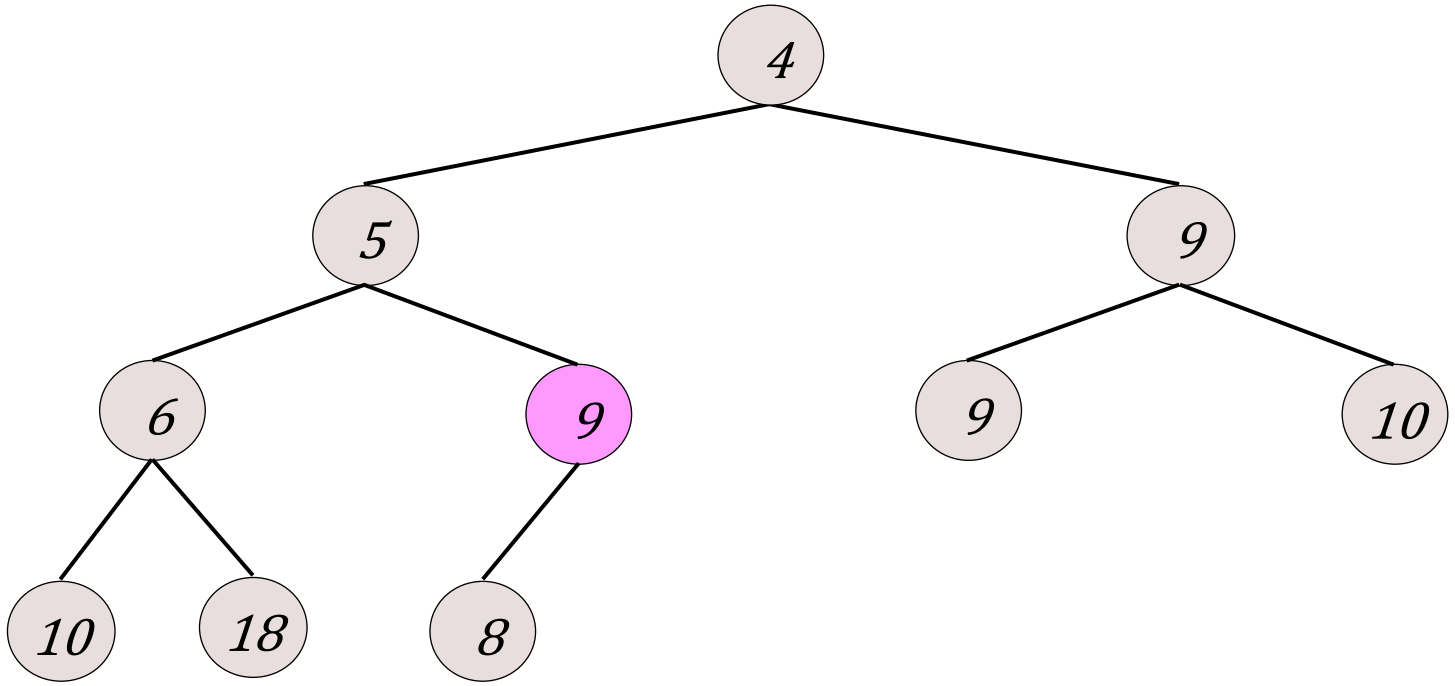


- Heapify the above min-heap into a max-heap

Example: Min-Heap → Max-Heap

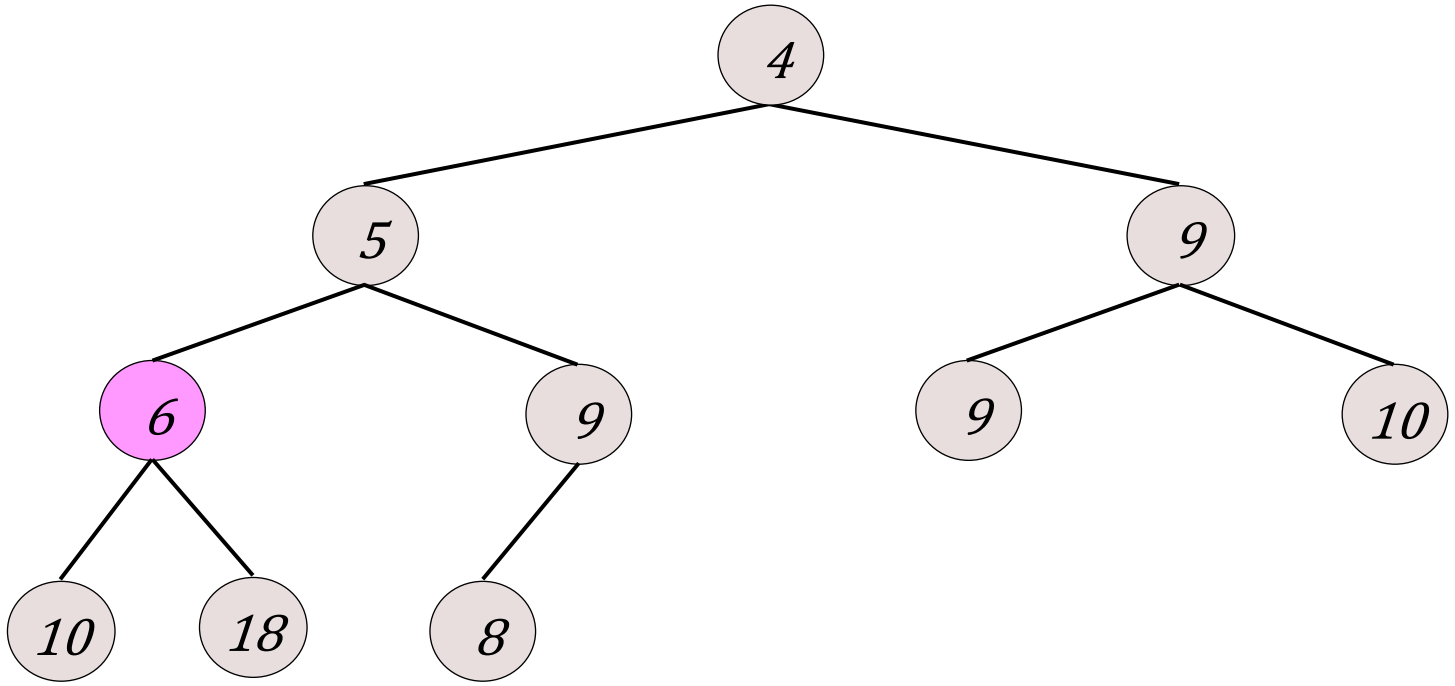


Example: Min-Heap → Max-Heap



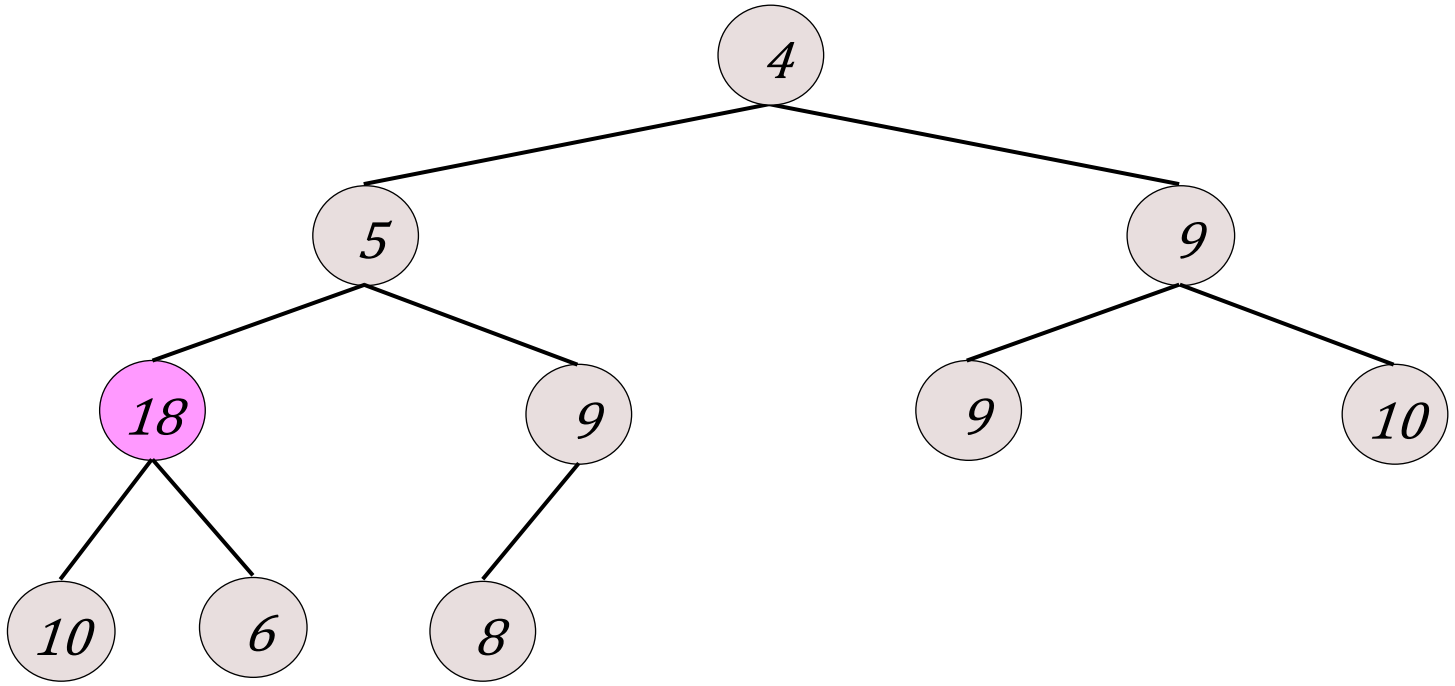
- Down-heap bubbling

Example: Min-Heap → Max-Heap

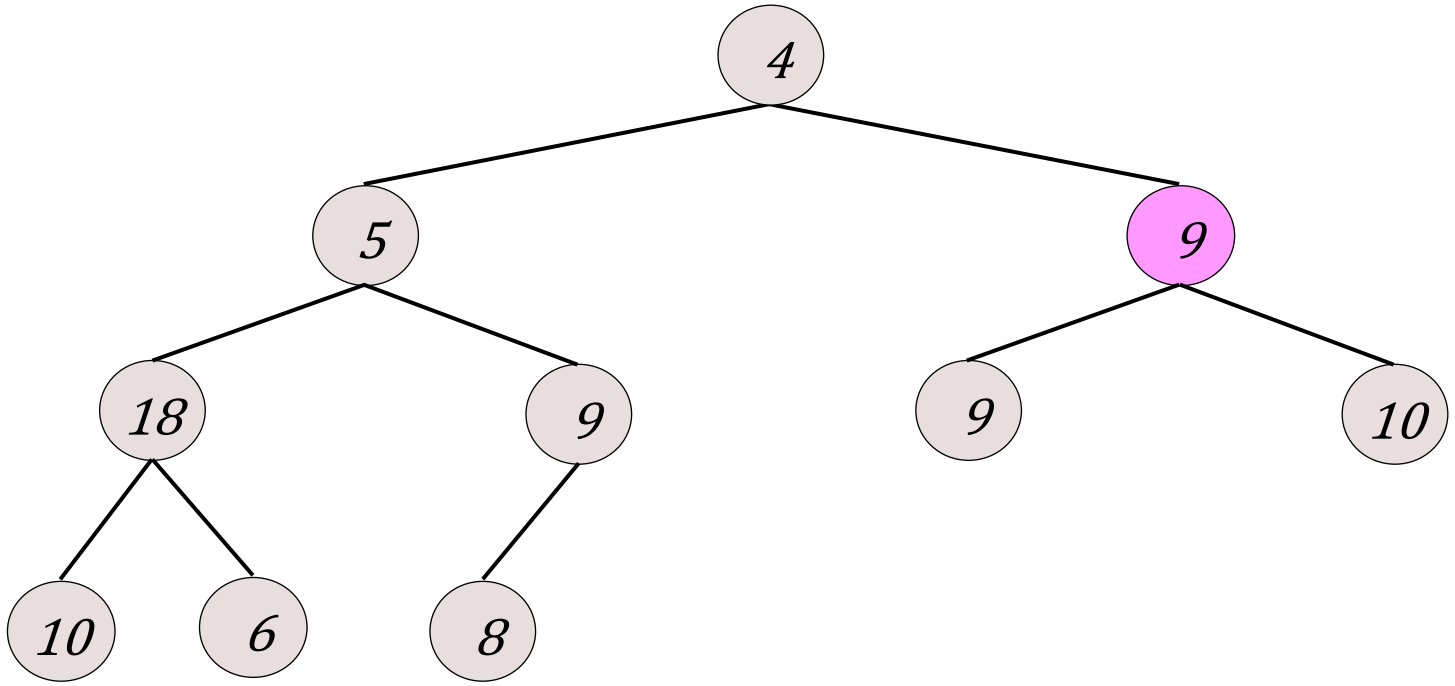


- Move to the next lower array position

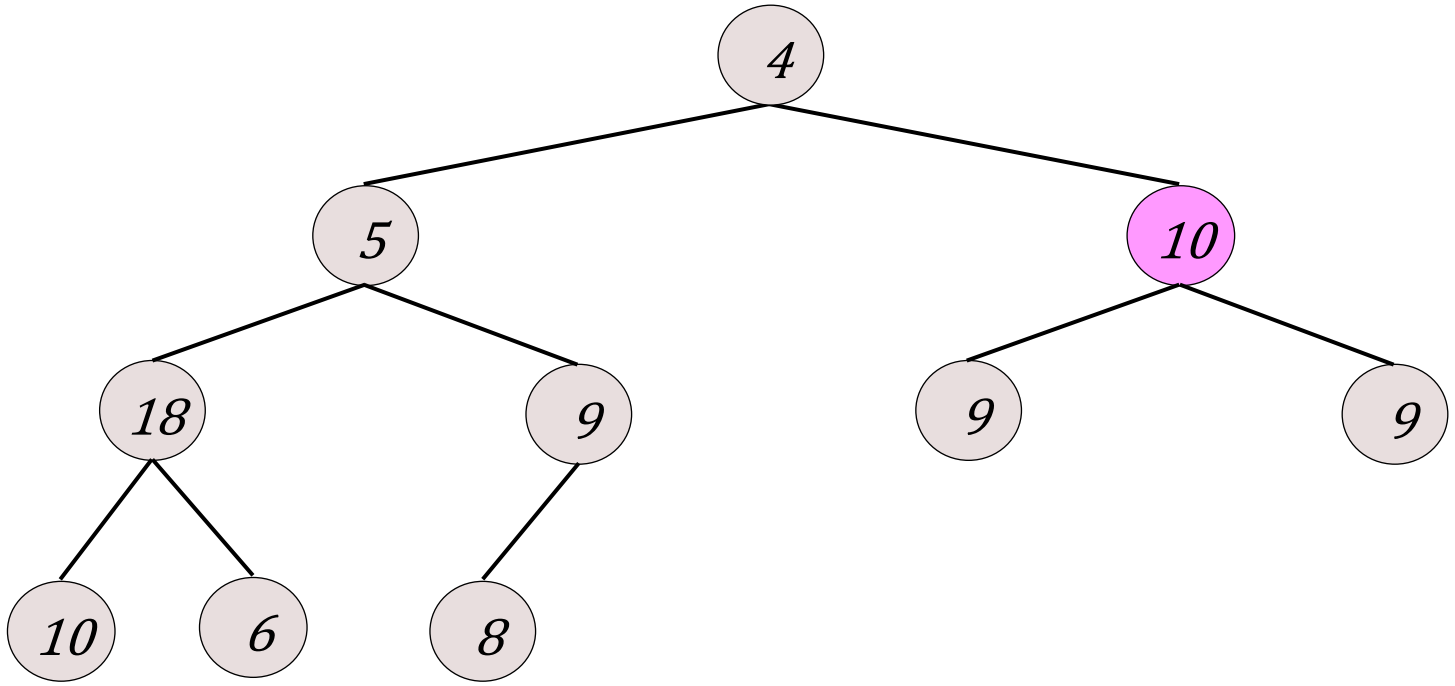
Example: Min-Heap → Max-Heap



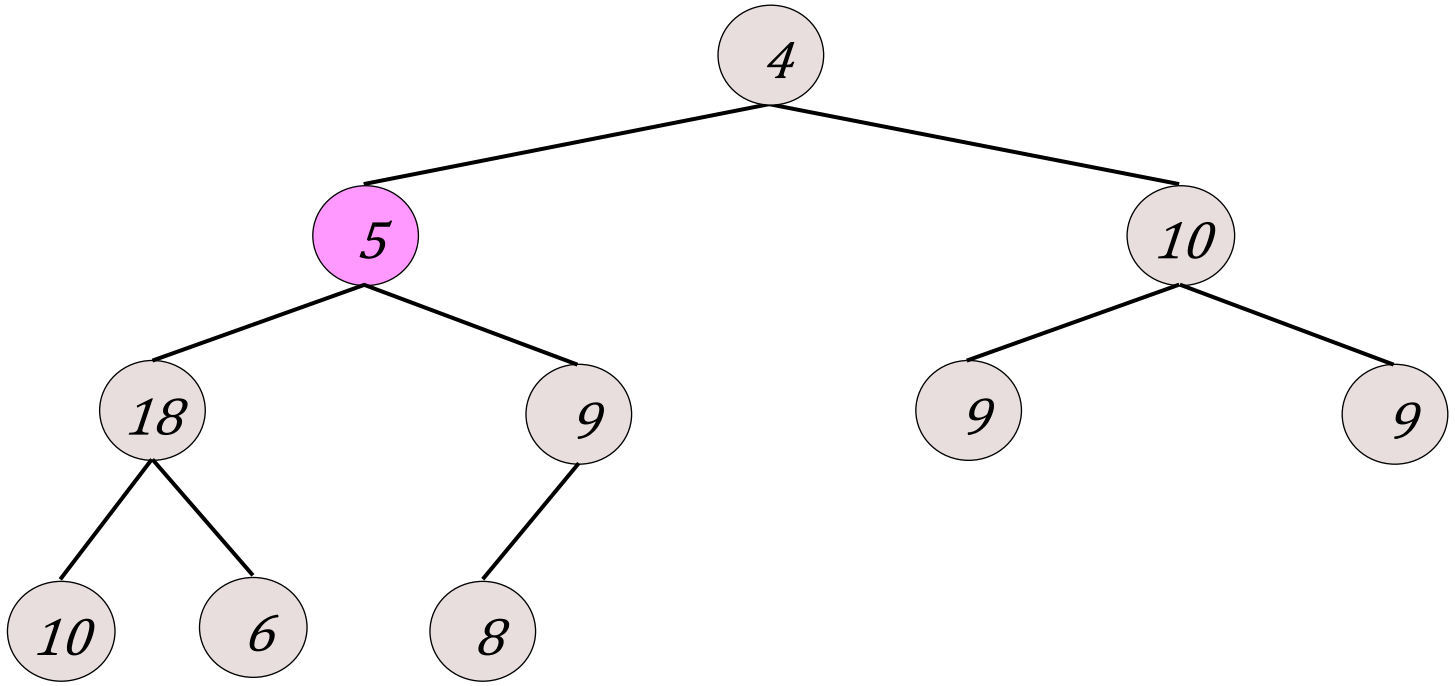
Example: Min-Heap → Max-Heap



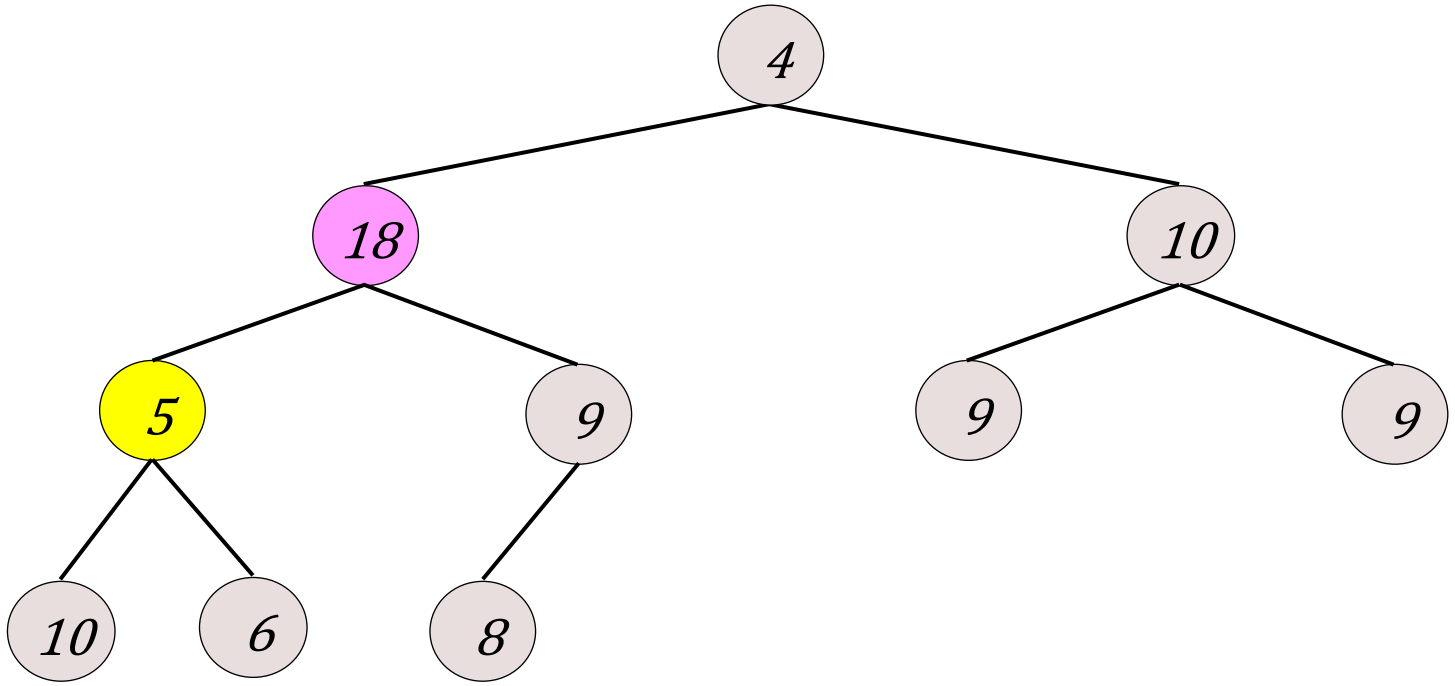
Example: Min-Heap → Max-Heap



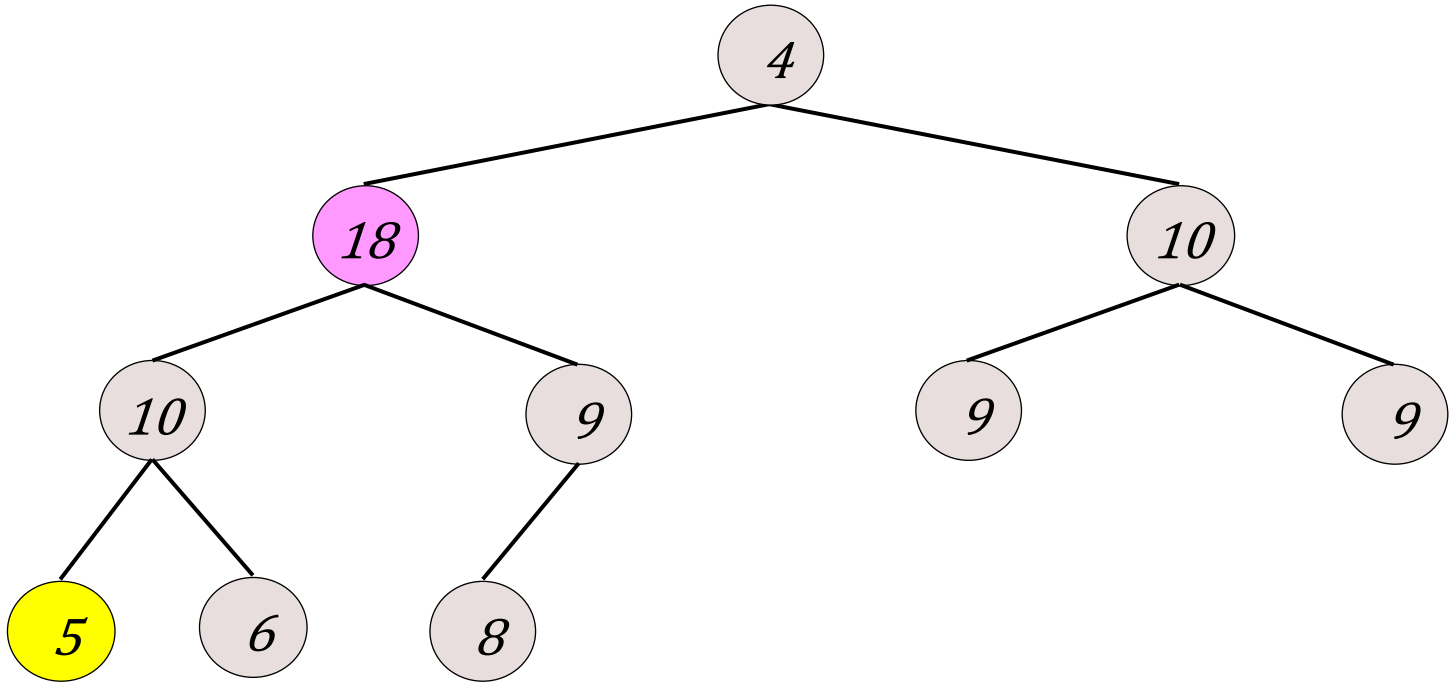
Example: Min-Heap → Max-Heap



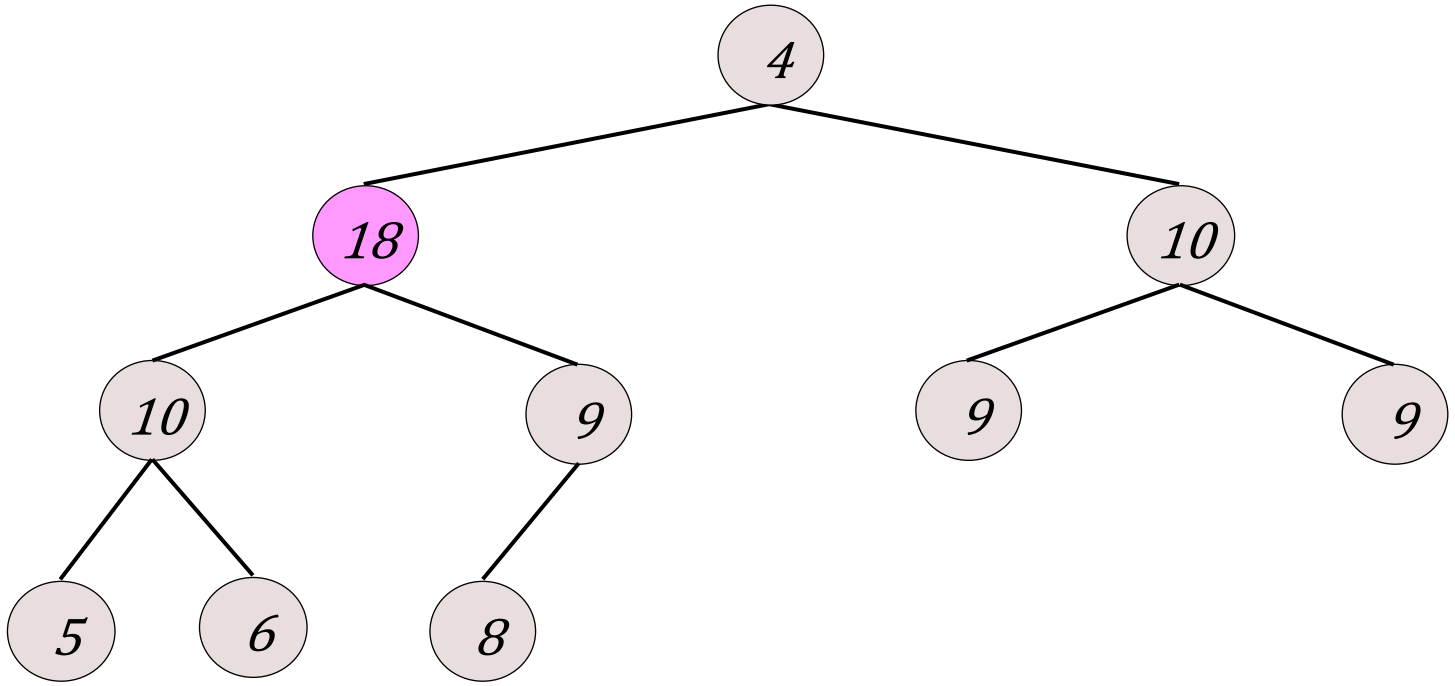
Example: Min-Heap → Max-Heap



Example: Min-Heap → Max-Heap

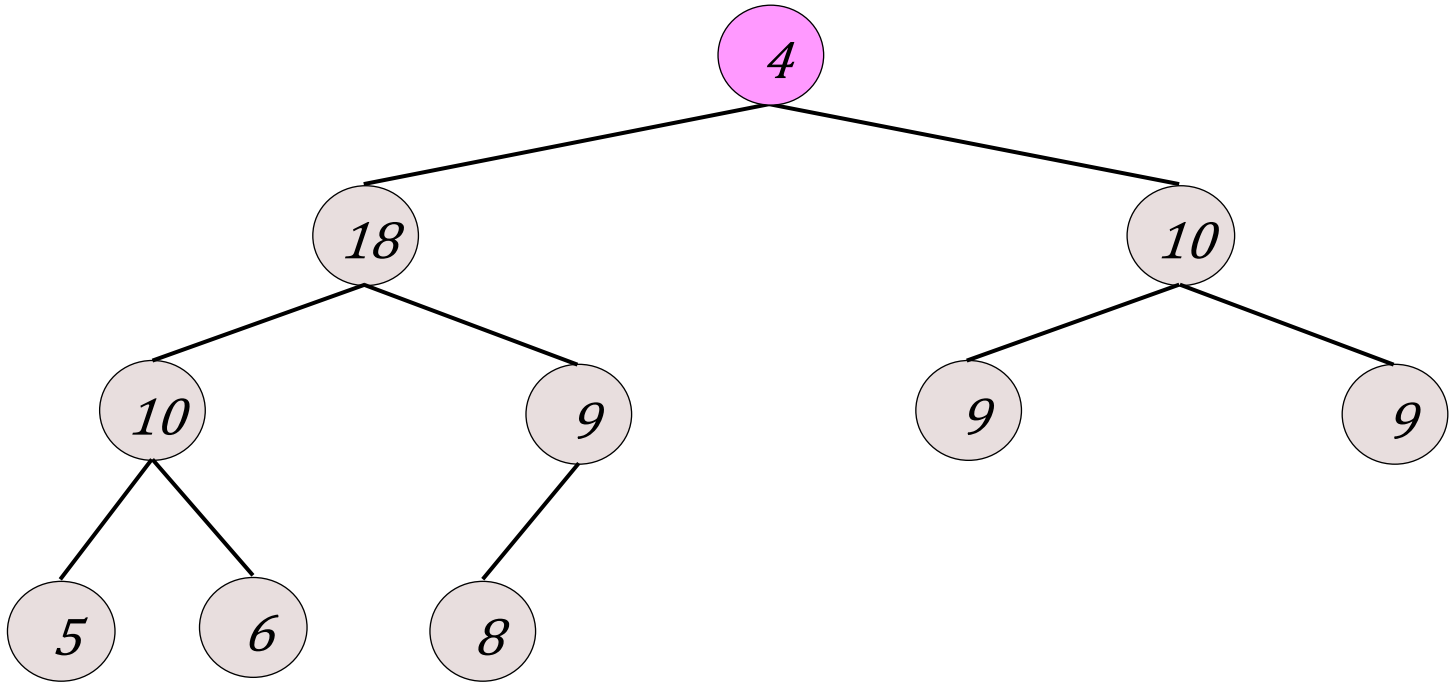


Example: Min-Heap → Max-Heap



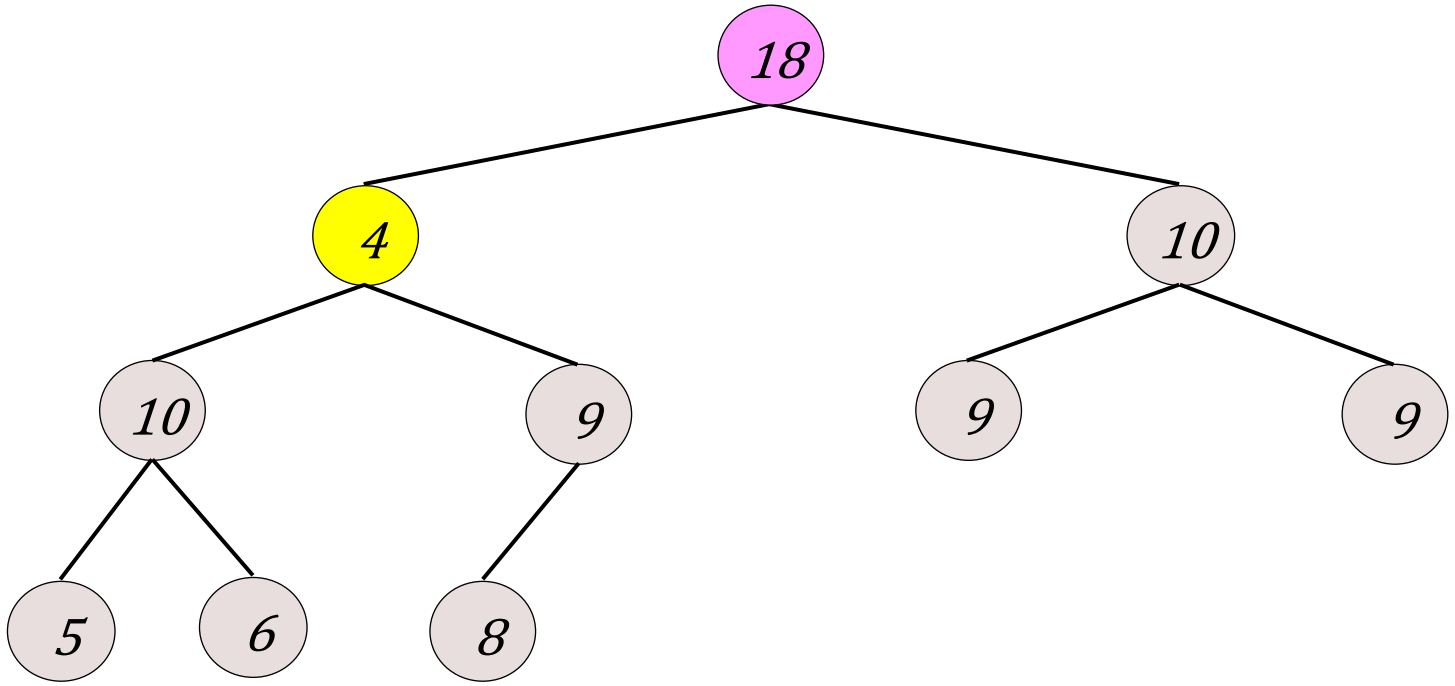
- Done for **5**

Example: Min-Heap → Max-Heap

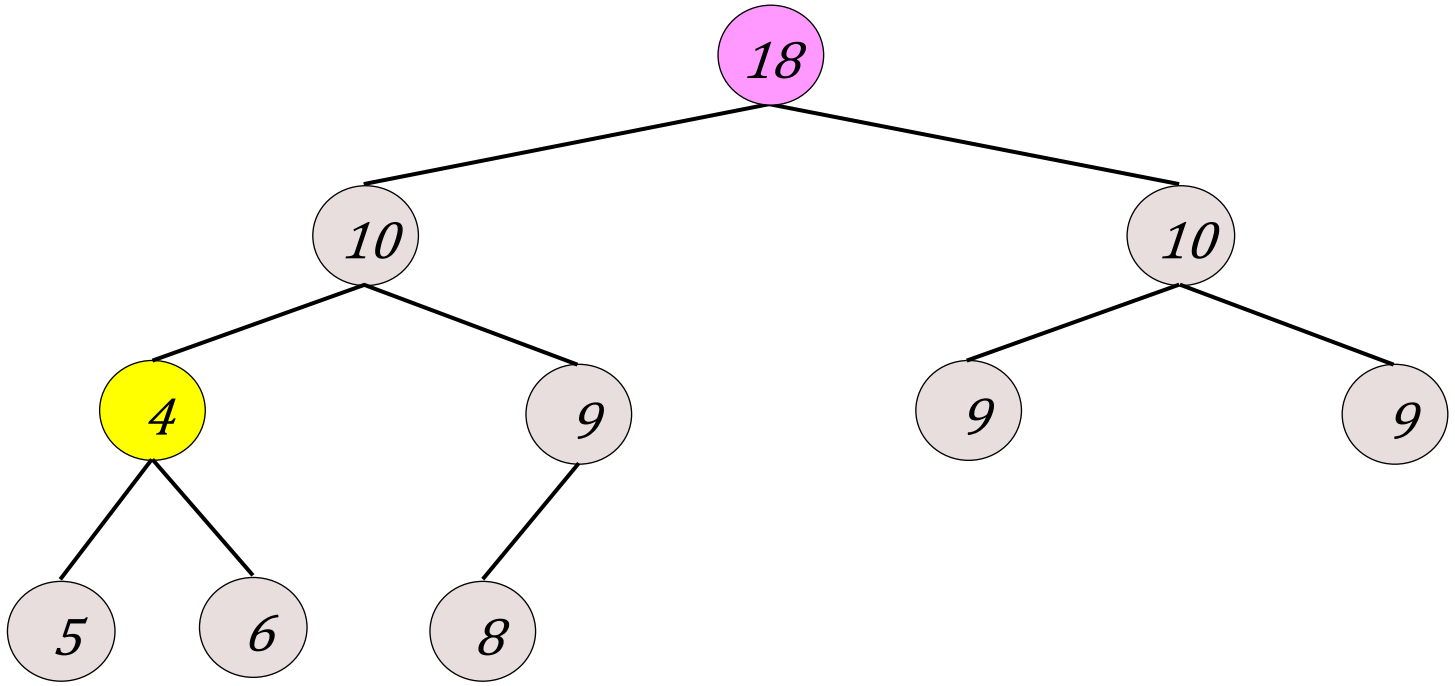


- Move to next lower array position

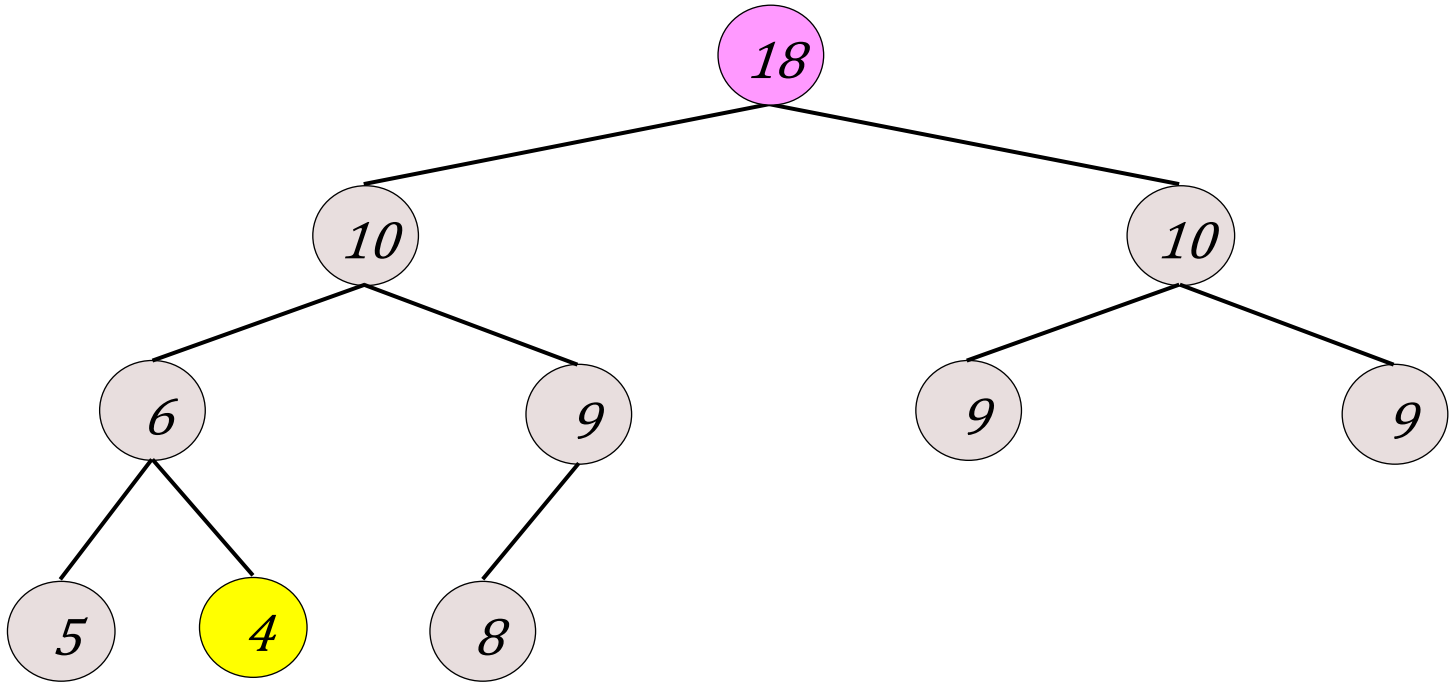
Example: Min-Heap → Max-Heap



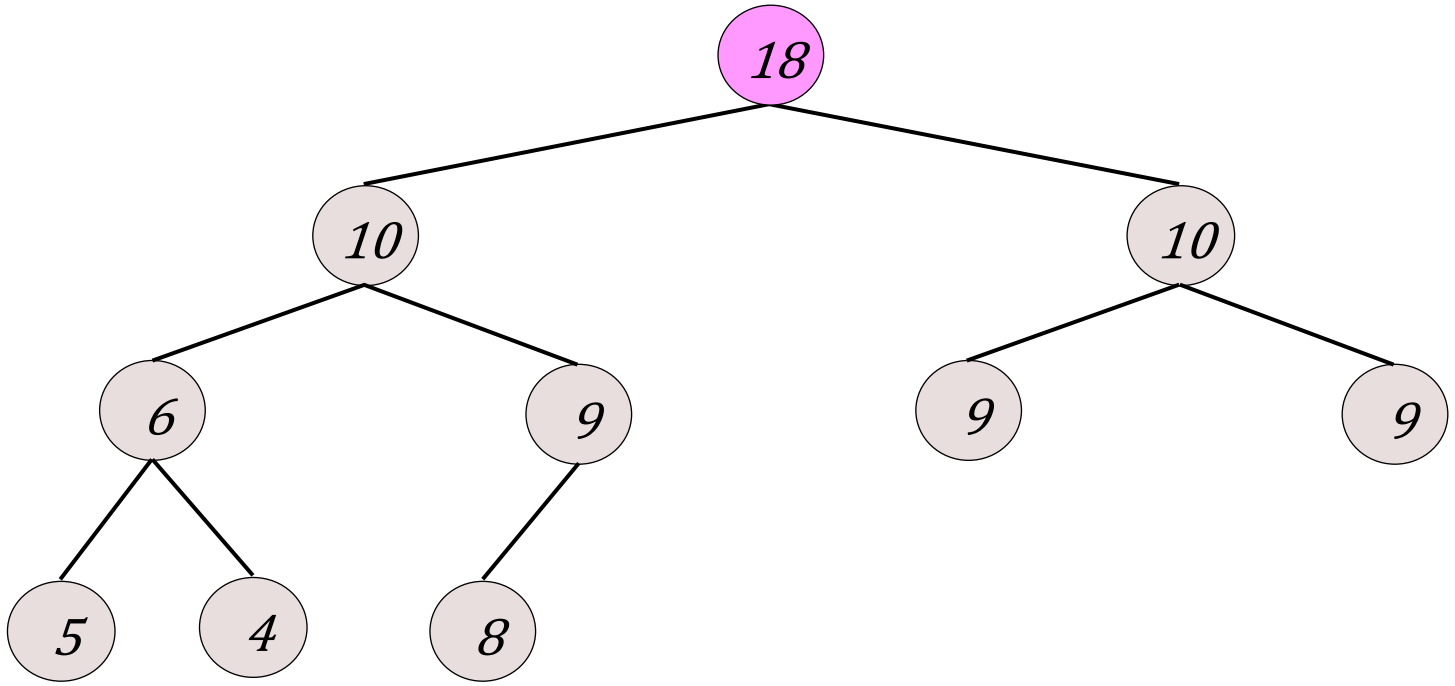
Example: Min-Heap → Max-Heap



Example: Min-Heap → Max-Heap

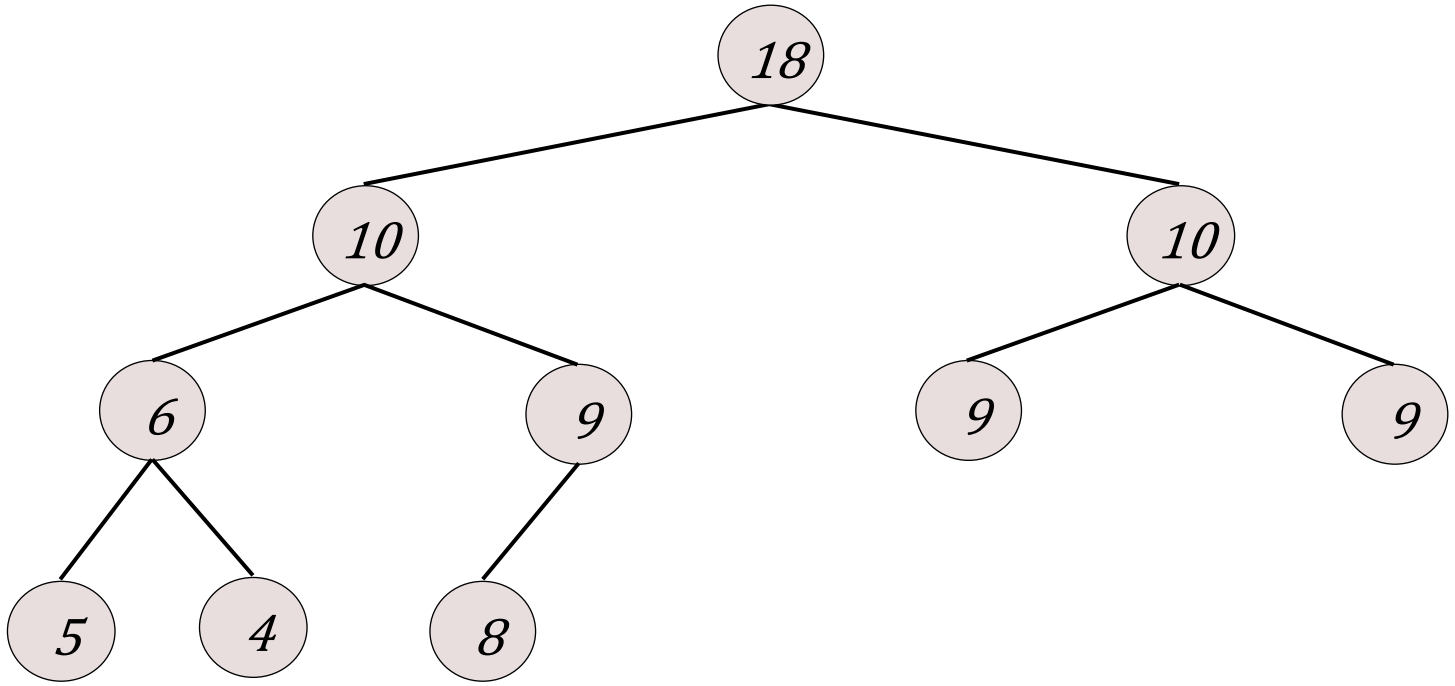


Example: Min-Heap → Max-Heap



- Done for 4

Example: Min-Heap → Max-Heap



- Done!

Quiz

- In the heapify algorithm, why do we start from the last non-leaf node instead of starting from the root?

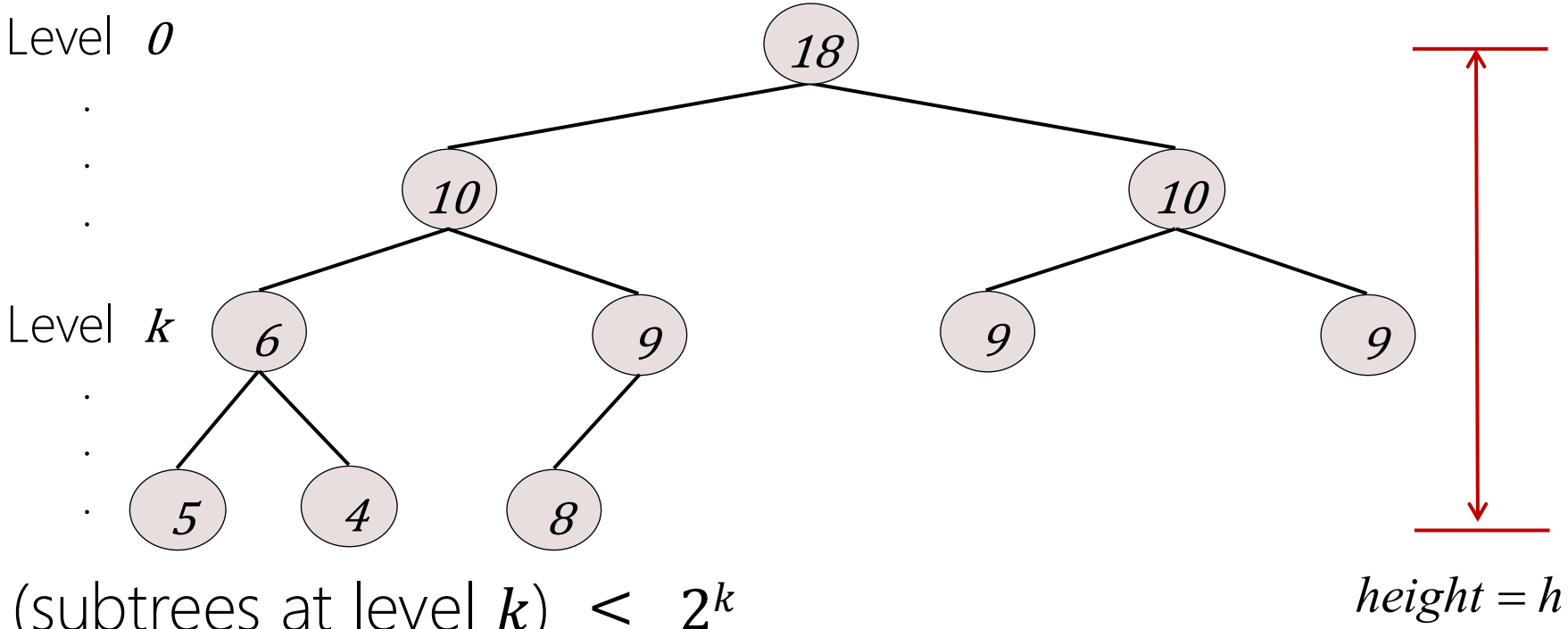
Recap: How to Heapify CBT

- How do we pick the *best* rearrangement

- Algorithm (from induction)
  - Input: Array-based CBT

1) Start with the rightmost array position k that has a child
   (NB: the last non-terminal node at index $k = floor(\frac{n}{2})$), n is # of nodes
2) If the subtree rooted at k is not a min-heap
   - Rearrange the subtree (by pushing down k until it reaches a level where it is less than its children, or is a leaf node)
3) Repeat the step-2 at $k-1, k-2, \ldots, 1$ (from the high index of the array to the low index)

Time Complexity of heapifying CBT



- Number (subtrees at level k) $\leq$ 2^k
- Time (each subtree) = $O(h - k)$
- Time (all subtrees at level k) $\leq 2^k\,(h - k)$
- Total time = $O(\sum_{k=0}^{h-1} 2^k (h-k)) = \ldots = O(2^h) = O(n)$

Quiz

- How would you define k-ary tree?

Quiz

- If a k-ary min-heap contains n elements, what is its height?

Quiz

- Why is DeleteMin more expensive in a k-ary heap as k increases?

Quiz

- If insert operations dominate and DeleteMin is rare, which K value is preferable?

Summary

- Priority Queue
- Heap
- Binary Heap
- Complete Binary Tree (CBT)
- Min-Heap / Max-Heap
- Insert (Up-Heap Bubbling)
- DeleteMin (Down-Heap Bubbling)
- Heapify / Heap-Building